Axion overview

Here are **system-level categories** that cover *every part* of what you described (idea → pipeline → Agent Kit zip → agent executes with accuracy/consistency).

1) Mission & Success Criteria

What the system is for, what “good” means.

- Purpose, scope (simple → complex)
- Quality bar (speed *and* quality)
- Non-goals (what the system is not)

2) Roles & Responsibilities

Who does what, and what each role is accountable for.

- User (provides inputs)
- System (produces planning artifacts)
- Agent (executes build using artifacts)
- Platform (where the agent runs)

3) Inputs Surface

Everything the system can accept.

- Project intake (form data)
- Optional add-ons (links, examples, constraints)
- Input validity rules (required/optional, format expectations)

4) Canonical Specification Model

The “shape” of truth the system builds internally.

- The minimal set of concepts that must exist for any project
- How those concepts relate (hierarchies + references)
- How ambiguity is represented (unknowns, assumptions)

5) Template System

How detailed guidance is standardized and reused.

- Template types (what kinds exist)

- Template variables (what gets filled)
- Template completeness rules (what must be specified)

6) Knowledge & Standards Library

Reusable standards the system injects into builds.

- Design standards (typography, components, layout rules)
- Engineering standards (stack choices, auth patterns, conventions)
- Quality standards (testing, security, performance expectations)

7) Pipeline Orchestration

How raw inputs become a complete build plan.

- Stages (transformations from intake → structured plan)
- Stage boundaries (what each stage must output)
- Ordering rules (dependencies between stages)

8) Assembly & Packaging

How outputs become the deliverable “Agent Kit.”

- Domain grouping model (how the plan is partitioned)
- Zip structure rules (where things live)
- Manifest/entrypoints (what the agent uses to start)

9) Agent Instruction Layer

How the agent is told to use the kit correctly.

- Prompt structure (how instructions are presented)
- Reading order rules (what to read first, second, etc.)
- Execution rules (how to apply the kit during building)

10) Execution Contracts

Rules that constrain the agent to prevent drift.

- “Must/never” rules during implementation
- Allowed vs disallowed behavior (assumptions, inventions, shortcuts)
- Change discipline (what can be modified and how)

11) Verification & Proof

How correctness is validated without argument.

- What checks exist (build/tests/contracts)
- What counts as proof
- Failure handling rules (stop, retry, escalate)

12) State & Continuity

How continuity is maintained across steps/sessions/platforms.

- What state must persist (progress, decisions, versions)
- Snapshot format (how “current truth” is recorded)
- Resuming rules (how an agent restarts without drifting)

13) Versioning & Evolution

How v2 stays stable over time.

- Version identifiers for kits/templates/standards
- Compatibility rules (old kits with new system, etc.)
- Deprecation and migration rules

14) Governance & Control

How you prevent the system from becoming “junkyard parts” again.

- Ownership of standards/templates
- Review/approval rules for changes
- Auditing (traceability of why something exists)

15) Tooling & Platform Integration

How the kit runs “anywhere.”

- Platform assumptions (file access, terminals, constraints)
- Integration adapters (what must change per platform)
- Minimum runtime requirements

Mission and success criteria

Mission & Success Criteria

Mission

Axion is a **documentation-and-template system** that enables an AI agent to turn an idea into a working product **much faster** while staying **accurate, consistent, and high-quality**.

Axion is **not** the agent. It is the **tooling layer** that does the heavy lifting for the agent by providing complete, structured guidance so the agent doesn't guess.

Core Objectives

1. **Speed with quality**
- Reduce time-to-product significantly **without** lowering build quality.
2. **Stability across complexity**
- Support projects ranging from very simple to highly complex **using the same system shape**, not a different process each time.
3. **Consistency**
- Produce repeatable, predictable results across runs, platforms, and agents.
4. **Low drift**
- Minimize agent "guessing," reinterpretation, and silent changes by making intent and standards explicit.

Success Criteria (Measurable)

These are the outcomes that define whether Axion is "working":

1. **Clarity of intent**
- The produced plan makes it unambiguous what is being built.
- A second agent (or the same agent later) would build the same thing from the kit.
2. **Completeness**
- The kit contains everything required to execute the build without needing to "invent" key decisions (stack, auth approach, UI standards, etc.).
3. **Correctness**
- The resulting product matches the intent expressed in the input.
- The build follows the defined standards and constraints.
4. **Continuity**
- Work can pause and resume with minimal re-explaining.
- Progress and decisions remain coherent across steps.

5. **Efficiency**
- The agent spends most of its time executing and verifying, not re-planning or re-interpreting.

Non-Goals (Explicit)

Axion is not trying to be:

- A general chatbot that improvises solutions without structure
- A fully autonomous “decide everything” builder that replaces product intent
- A system that optimizes speed by skipping verification or standards
- A one-stack-only rigid pipeline that can’t scale in scope and complexity

---

Quality Bar (What “High Quality” Means)

A build is high-quality when it is:

- **Consistent** (follows defined standards)
- **Verifiable** (can be proven correct via checks)
- **Maintainable** (organized, understandable, structured)
- **Secure enough for the intended context** (auth/data practices match the intended use)
- **User-intent aligned** (build matches what the user asked for)

Primary Failure Conditions (What counts as “Axion failed”)

- The agent had to guess core requirements or standards
- The kit produces contradictory guidance
- The output product differs materially from the intended product
- The process cannot reliably resume after interruption
- The system only works for “simple” projects but breaks down as complexity rises

Roles & Responsibilities

Roles & Responsibilities

1) User (rephrased)

Responsibility: Provide as much product detail as possible through a required intake form so Axion can capture intent with minimal ambiguity.

- Complete the required project intake form thoroughly (features, users/roles, workflows, constraints, preferences, examples)
- Provide any supporting references (similar products, design/style direction, links, documents, existing repo context)
- Clarify priorities and tradeoffs when necessary (speed vs depth, MVP vs full build, “must-have” vs “nice-to-have”)

Accountable for: The completeness and accuracy of the input intent and constraints Axion will formalize into a build plan.

---

2) Axion System (the tool / pipeline)

Responsibility: Convert the user’s intent into an execution-grade build plan that minimizes agent guessing.

- Normalize input into structured specifications
- Apply standards and rules consistently (design, engineering, security, quality)
- Produce a complete “Agent Kit” package that is internally coherent
- Produce the agent-facing instructions that tell the agent how to use the kit

Accountable for: Clarity, completeness, consistency, and packaging correctness.

---

3) AI Agent (builder/executor)

Responsibility: Implement the product according to the kit, with discipline.

- Read and follow the kit in the required order
- Implement exactly what the kit specifies (no silent reinterpretation)
- Run checks and provide proof of completion
- Record progress/state in the required format so work can resume cleanly

Accountable for: Faithful execution + verification.

---

4) Vibe-coding Platform / Runtime (where the agent runs)

Responsibility: Provide the working environment.

- File system access (read/write project files)
- Terminal/runtime (install, build, test, run)
- Tool permissions and limitations (what the agent can/can't do)

Accountable for: Execution capability and constraints (not correctness of the plan).

---

5) System Owner / Maintainer (you)

Responsibility: Keep Axion "manufactured," not "junkyard."

- Maintain standards, rules, and templates
- Control changes and versioning
- Ensure internal consistency across the system
- Define the quality bar and what constitutes proof

Accountable for: Long-term stability and evolution of the system.

Inputs Surface

Inputs Surface

Definition

The Inputs Surface is **everything Axion can accept** from a user (and optionally from an existing project environment) to accurately capture intent and constraints before generating the build plan.

---

1) Required Input Channel

Project Intake Form (mandatory)
This is the baseline input that every project must provide. Its job is to capture enough detail that the system can produce a build plan with minimal guessing.

What it must cover at minimum:

- What is being built (product concept and scope)
- Who it's for (target users + roles)
- What it must do (features + workflows)
- What data exists (entities / objects / content types)
- Constraints (must-haves, must-not-haves, compliance, timeline priorities)
- Preferences (design direction, platform targets, performance expectations)

---

2) Optional Input Channels

These are allowed but not required; they reduce ambiguity and improve fit.

- **References:** example apps/sites, inspiration, competitors, screenshots
- **Brand/Design Direction:** typography preferences, color direction, layout vibe, UI references
- **Technical Constraints:** required stack, hosting, auth requirements, third-party services
- **Policy/Rules Constraints:** moderation rules, safety constraints, permissions model
- **Success Criteria:** what "done" means, acceptance expectations

---

3) Existing Project Inputs (if applicable)

If the idea is building on something that already exists, Axion can accept context from that environment.

Examples:

- Existing repository structure
- Existing data model or API shape
- Current tooling constraints (CI, linting, formatting rules)
- Current deployment/runtime constraints

---

4) Input Quality Rules (what makes inputs usable)

- **Required fields must be present**
- **Contradictions must be resolved** (or explicitly marked as unresolved)
- **Ambiguity must be representable** (unknowns clearly labeled as unknowns, not guessed)
- Inputs should be **structured** whenever possible (lists, options, role definitions, feature checklists)

---

5) Input Output Boundary

The Inputs Surface ends when the system can produce:

- a clear, internally consistent understanding of the product intent, and
- enough constraints/standards to generate a build plan without inventing core decisions.

Skill Level (Depth Profile)

Definition: A required routing input that determines how “guided vs detailed” the intake experience is, affecting module depth, requiredness, UI scaffolding, and autofill support.

Allowed values (locked):

- **Beginner (Guided / Spoon-fed):**
    - Progressive disclosure (fewer sections visible at once)
    - More helper text/examples, more pick-from-options, stronger defaults
    - Reduced required depth in advanced subforms (simplified data/API/etc.)

    - Autofill supported and optimized to help completion (user-controlled)

- **Intermediate (Balanced):**
    - Standard module set for selected preset
    - Mix of structured pickers and free-text
    - Moderate required depth (roles, workflows, core data objects, basic constraints)
    - Advanced sections available; required only when triggered by gates
- **Expert (Maximum Control / Full Detail):**
    - Full module set + advanced subforms enabled
    - Minimal hand-holding, maximum configurability
    - Higher required thresholds (more workflows, deeper data model, explicit constraints)
    - More explicit verification expectations and technical detail fields

What it affects:

- Which modules/pages are rendered (and which are hidden or collapsed)
- Required field thresholds (e.g., number of workflows, depth of object definitions)
- UI scaffolding (helper text, examples, defaults)
- Autofill behavior (drafting level and where it's recommended)

Output: included in the routing object as `skill_level`, consumed by form rendering logic and downstream planning.

Canonical Specification Model

Canonical Specification Model

Definition

The Canonical Specification Model is the system's **single structured "truth format"** that all inputs are converted into before any build planning or packaging happens. Its job is to make intent **unambiguous, referencable, and consistent** so an agent can execute without guessing.

---

1) What it must represent (core concept types)

A complete spec must be able to represent:

- **Product Scope**
  - what exists in the product (and what explicitly doesn't)
- **Users & Access**
  - user types/roles
  - permissions and restrictions
- **Capabilities**
  - features (what the product can do)
  - workflows (how users accomplish things step-by-step)
- **Data**
  - data objects / records / content types
  - relationships between them
  - required fields vs optional fields
- **Interfaces**
  - screens/pages/views (what users see)
  - actions on those screens (what users can do)
  - APIs/endpoints or internal operations (what the system does)
- **Rules**
  - business rules (what must always be true)
  - validation rules and edge cases
  - "must/never" constraints
- **Standards**
  - design standards
  - engineering standards
  - security/quality standards
- **Acceptance**
  - what counts as correct/complete
  - how correctness is checked

---

2) Structural rules (how the model stays stable)

- **Stable IDs:** every major item (feature, screen, data object, rule) must be referencable by an ID so nothing is “hand-wavy.”
- **Single definition:** each concept is defined once and referenced elsewhere (prevents conflicting duplicates).
- **Explicit unknowns:** unknown or undecided items must be stored as unknowns, not filled by assumptions.
- **Separation of concerns:** “what it is” (definition) is separate from “how it’s built” (implementation details).

---

3) Minimal completeness rules (what makes it “execution-grade”)

A spec is execution-grade when it includes:

- At least one defined **primary user** and their core workflow(s)
- A defined set of **capabilities** tied to workflows
- Defined **data objects** required to support those workflows
- Defined **access rules** for who can do what
- Defined **standards** that constrain decisions the agent would otherwise guess
- Defined **acceptance checks** that can prove completion

---

4) Output of this section (what the model enables)

Once everything is in this Canonical Spec format:

- planning can be generated consistently,
- templates can be filled deterministically,
- packaging can be predictable,
- and agent execution becomes “follow instructions,” not “interpret intent.”

Depth Profile and Spec Completeness

Definition: The Canonical Spec supports a *single stable structure* across all projects, but its **required completeness thresholds** vary by `skill_level` to balance usability (Beginner) with precision (Expert).

What remains constant (structure invariants)

- Same concept types exist in the model (roles, workflows, capabilities, data objects, rules, etc.)
- Stable IDs and references are always required
- Unknowns are always represented explicitly (never guessed)
- No duplicate truth (define once, reference everywhere)

What varies by skill level (completeness requirements)

Beginner

- Requires a minimal execution-grade core:
    - at least 1 role
    - at least 1 must-have capability set
    - at least 3 core workflows
    - minimal data object definitions (names + basic purpose) when applicable
- Allows more items to remain as explicit `unknown` without blocking kit generation.

Intermediate

- Requires the standard execution-grade core:
    - roles with baseline permission intent
    - workflows tied to roles
    - initial data objects with key fields (when applicable)
    - basic rules/constraints captured
- Unknowns are allowed, but fewer than Beginner; more must be concretely specified.

Expert

- Requires high specificity:
    - deeper workflow detail (more workflows and/or deeper step detail)
    - explicit data objects + key fields + relationships where relevant
    - explicit constraints and edge cases
    - clearer verification expectations
- Unknowns are allowed only when explicitly justified and isolated (should not affect core build decisions).

Spec validation rule

The spec is considered “valid” when it meets:

- **Structural validity** (always required), and
- **Profile completeness** (Beginner/Intermediate/Expert thresholds)

Template System

Template System

Definition

The Template System is the **library of structured document patterns** used to turn the Canonical Specification into **execution-ready instructions**. Templates exist so the agent doesn't have to guess formatting, completeness, or standards—Axion "manufactures" the guidance the agent follows.

---

1) Template Types (what categories of templates exist)

Templates should cover every major output class the system needs to guide a build:

- **Planning Templates**
  - project plan structure, milestones, task breakdowns
- **Design Templates**
  - UI rules, typography/layout decisions, component usage rules
- **Architecture Templates**
  - system structure, module boundaries, data flow, integration points
- **Implementation Templates**
  - build steps, coding conventions, file structure rules, patterns to apply
- **Security & Access Templates**
  - auth/permissions rules, data handling rules, risk constraints
- **Quality Templates**
  - tests, acceptance criteria, verification requirements, performance checks
- **Operational Templates** (if needed)
  - deployment constraints, environments, monitoring/logging expectations

---

2) Template Structure (how a template is built)

Every template should have a consistent internal structure so it's predictable for an agent:

- **Purpose** (what this template is used for)
- **Required fields** (what must be filled)
- **Optional fields** (what improves quality but isn't mandatory)
- **Rules** (what must always be true inside this template)
- **Output format** (how it should be written/organized)
- **Cross-references** (what other items it links to by ID)

---

3) Template Variables (how templates get filled)

Templates should be fillable using data pulled from the Canonical Spec:

- **Direct inserts:** values copied from the spec (roles, features, data objects)
- **Derived inserts:** values computed from relationships (feature → screens → operations)
- **Standard inserts:** values injected from the standards library (design rules, stack rules)

Key requirement: a template must never require “creative guessing” to fill.

---

4) Completeness Rules (when a filled template is “valid”)

A filled template is valid when:

- all required fields are populated
- no contradictions exist within the template
- referenced items exist (no broken references)
- any unknowns are explicitly labeled (not silently resolved)

---

5) Consistency Rules (how templates avoid junkyard growth)

- **One purpose per template:** avoid mixing concerns
- **No duplicate truth:** templates reference canonical items rather than redefining them
- **Standardized naming:** every template follows the same naming and section patterns
- **Versioned templates:** changes are controlled so old kits don’t break

---

6) Template Output Boundary

The Template System outputs a set of filled, coherent documents that:

- fully constrain key build decisions,
- provide stepwise guidance,
- and can be assembled into a packaged kit without missing pieces.

Depth Profile Application to Templates

Definition: Templates remain structurally consistent across projects, but **which sections are required and how much detail is expected** varies by `skill_level`. This prevents Beginner users from being forced into expert-level specification while still producing usable guidance.

How `skill_level` affects templates

1. **Required sections**

- Beginner: only the essential sections are required for execution; advanced sections may be omitted or marked “optional.”
- Intermediate: standard required sections for the selected preset.
- Expert: advanced sections become required where applicable (constraints, edge cases, verification detail, etc.).

2. **Granularity thresholds**

- Beginner: fewer required items (fewer workflows, fewer field-level details).
- Intermediate: moderate.
- Expert: higher minimum counts and deeper detail.

3. **Unknown handling**

- Beginner: templates may contain more `unknown` placeholders; they must be clearly labeled and consolidated.
- Intermediate: fewer unknown placeholders; more must be resolved.
- Expert: unknown placeholders should be rare and justified; they must not exist in core decision areas.

4. **Autofill interaction (if enabled)**

- Autofill populates template fields as **DRAFT**, respecting `skill_level`:
    - Beginner: more drafting to reduce friction
    - Intermediate: moderate drafting
    - Expert: minimal drafting unless high-confidence

Implementation note (system behavior, not UI)

Templates can support this in either of two ways:

- **Single template with requiredness rules** per section/field, keyed by `skill_level`, or
- **Template variants** per skill level that share the same structure but differ in required fields.

Either way, the output must remain consistent and machine-navigable.

Knowledge & Standards Library

Knowledge & Standards Library

Definition

The Knowledge & Standards Library is the system's **central store of reusable rules and defaults** that get applied across projects so the agent doesn't re-decide basics every time. It exists to enforce **consistency, quality, and predictability**.

---

1) What it contains (standard categories)

- **Design Standards**
  - typography rules (font choices, sizes, hierarchy)
  - spacing/layout rules
  - component style rules (buttons, forms, cards, navigation patterns)
  - accessibility rules (minimums)
- **Engineering Standards**
  - stack defaults (frameworks, languages, tooling)
  - project structure conventions
  - coding conventions (naming, formatting, error handling)
  - dependency rules (what's allowed vs discouraged)
- **Security Standards**
  - authentication patterns (session/token approach, password policy, etc.)
  - authorization approach (roles/permissions enforcement rules)
  - data handling rules (PII, encryption, logging constraints)
  - rate limits / abuse controls (if applicable)
- **Quality Standards**
  - required checks (build, lint, tests)
  - test expectations (unit/integration minimums where applicable)
  - performance expectations (baseline targets if any)
  - documentation expectations (what must be documented)
- **Product/UX Standards** (if you want them global)
  - navigation conventions
  - error states and empty states
  - onboarding patterns
  - copy/voice rules (tone of UI text)

---

2) How it is applied

The library is used in three ways:

- **Defaults:** automatic choices used unless overridden by the project input
- **Constraints:** non-negotiable requirements (must/never rules)
- **Recommendations inside the kit:** standardized options presented as the chosen system pattern

---

3) Override rules (to stay flexible)

To support simple → complex projects:

- Standards must define what is **fixed** vs **configurable**
- If configurable, they must define **allowed options**
- Overrides must be explicit, traceable, and consistent across the kit

---

4) Consistency guarantees

A stable library should ensure:

- The same input yields the same applied standards (deterministic)
- Two projects using the same standards version behave consistently
- No “local template drift” where one document contradicts another

---

5) Versioning expectations

Because standards evolve:

- The library must be versioned
- Every generated kit must record which standards version it used
- Changes must not silently alter old kits

---

6) Output boundary

The Knowledge & Standards Library outputs **a resolved set of standards for a specific project** (defaults + overrides) that downstream planning/templates can apply consistently.

Pipeline Orchestration

Pipeline Orchestration

Definition

Pipeline Orchestration is the **end-to-end staged process** that takes user inputs and transforms them into a complete, internally consistent build plan and packaged kit. Its purpose is to make the transformation **repeatable, auditable, and deterministic** instead of ad-hoc.

---

1) Stage Model (what a “pipeline” is made of)

The pipeline is a sequence of stages where each stage has:

- **Inputs** (what it consumes)
- **Transform** (what it does)
- **Outputs** (what it must produce)
- **Validation** (how it proves it did it correctly)

Stages should be designed so each one can be evaluated independently.

---

2) Typical Stage Responsibilities (high level)

These are the kinds of responsibilities the pipeline must cover, regardless of implementation:

1. **Intake & Normalization**
- convert form responses into structured data
- standardize wording, formats, naming
- detect missing required info
2. **Conflict & Ambiguity Handling**
- identify contradictions
- mark unresolved items explicitly
- apply “unknown” representations rather than guessing
3. **Standards Resolution**
- load global standards
- apply project overrides
- resolve final “ruleset” for this project
4. **Canonical Spec Construction**
- create the single structured truth model
- assign stable IDs to key items
- enforce relationship integrity

5. **Plan Generation**
- decompose the spec into actionable build slices
- map capabilities → interfaces → operations → data
- produce build sequencing and dependencies

6. **Template Filling**
- populate all required documents from the canonical spec + standards
- enforce completeness rules per template

7. **Packaging**
- assemble outputs into the kit structure
- generate manifests/entrypoints for agent consumption

---

3) Orchestration Rules (what keeps it stable)

- **Determinism:** same input + same standards version ⇒ same outputs
- **No hidden decisions:** every major decision must be either input-driven or standards-driven
- **Traceability:** outputs must be traceable back to input fields and/or standards
- **Fail fast:** if required info is missing, the pipeline should stop or mark clearly rather than improvise

---

4) Pipeline Outputs (what it guarantees at the end)

At completion, the pipeline must output:

- a complete build plan with consistent internal references
- a set of filled guidance documents
- a packaged kit that can be used in an external build environment
- a clear execution entrypoint for the agent

---

5) Pipeline Boundaries

Pipeline Orchestration ends when:

- the kit is assembled and ready to be consumed by an agent, and
- all validations for required completeness/consistency have passed.

Skill Level as a Routing Input

Definition: `skill_level` is a required routing parameter produced at intake that influences how the pipeline interprets, completes, and validates project input.

Where it enters the pipeline

- Consumed immediately after intake validation as part of the routing object:
    - `{ skill_level, category, type_preset, build_target, audience_context, autofill }`

What it changes in orchestration

1. **Normalization strategy**

- Beginner: stronger normalization (more standardization), more tolerant parsing of loose answers
- Intermediate: standard normalization
- Expert: preserve precision, fewer “helpful rewrites,” keep technical specificity intact

2. **Completeness thresholds**

- Beginner: lower minimum thresholds; allow more “unknowns” to be carried forward explicitly
- Intermediate: standard thresholds
- Expert: higher thresholds; require more explicit detail before packaging

3. **Module activation defaults**

- Beginner: default to fewer advanced modules unless gates require them
- Intermediate: include standard modules for the selected preset
- Expert: include advanced modules by default for the preset

4. **Autofill behavior (if enabled)**

- Beginner: draft more scaffolding content (clearly labeled DRAFT)

- Intermediate: draft only high-confidence sections
- Expert: draft minimal or skip drafting unless requested; prioritize accuracy over “helpful filling”

5. **Validation strictness**

- Beginner: strict on required basics, flexible on advanced specificity
- Intermediate: balanced
- Expert: strict on both basics and advanced sections that are enabled

Pipeline invariants (unchanged across skill levels)

- No invented requirements
- Unknowns must remain explicitly marked
- Deterministic outputs given same inputs + versions
- Proof/verification requirements remain enforced (though breadth may differ by profile)

Assembly & Packaging

Assembly & Packaging

Definition

Assembly & Packaging is the part of the system that takes all pipeline outputs and **manufactures the deliverable "kit"** in a consistent structure so an agent can load it, navigate it, and execute it without confusion.

---

1) Packaging Goals

- **Predictable structure:** the kit looks the same every time
- **Fast orientation:** the agent immediately knows where to start
- **Separation of concerns:** planning vs rules vs specs vs checks are not mixed
- **Portability:** the kit works across different build platforms

---

2) Kit Structure Model (what the package must include)

A complete kit must include, at minimum:

- **Entry Point**
  - the single starting artifact that tells the agent what to do first and how to use the kit
- **Resolved Spec**
  - the execution-grade "truth" representation for the product (the structured plan the rest references)
- **Guidance Documents**
  - filled templates that translate the spec into actionable instructions
- **Work Breakdown**
  - how the build is partitioned (modules/domains/sections) with clear boundaries and dependencies
- **Standards Snapshot**
  - the exact standards/rules used (and their version) so behavior is reproducible
- **Verification Artifacts**
  - what checks must be run and what constitutes proof
- **Manifests / Indexes**
  - maps that let the agent quickly find things (what exists, where it lives, how it connects)

---

3) Grouping Strategy (how outputs are organized inside the kit)

The kit needs a consistent method for grouping content, such as:

- by major product area (logical partitions)
- by build sequence (phases/slices)
- by responsibility type (specs vs instructions vs checks)

The key is that the grouping method is:

- consistent across projects
- navigable without reading everything
- able to scale with complexity

---

4) Packaging Rules (what must always be true)

- **Single entrypoint:** no ambiguity about where the agent starts
- **No orphan docs:** everything included must be referenced by a manifest/index
- **No duplicate truths:** the same decision should not exist in two places
- **Stable paths:** file locations should not vary between kits
- **Version stamps:** kit version + standards version are recorded in the kit

---

5) Portability Constraints

Because kits are unzipped and used in different environments, packaging must avoid assumptions like:

- specific absolute paths
- platform-specific tooling unless declared
- hidden dependencies not included or explicitly required

---

6) Output Boundary

Assembly & Packaging is complete when:

- the kit can be unzipped into an environment,
- the agent can follow the entrypoint and find everything it needs,
- and the structure is consistent and self-describing through manifests/indexes.

Agent Instruction Layer

Agent Instruction Layer

Definition

The Agent Instruction Layer is the **agent-facing guidance** that tells the AI agent exactly how to use the kit to build the product with high accuracy, consistency, and efficiency. Its job is to control agent behavior and reading order so execution is disciplined rather than improvised.

---

1) Core Responsibilities

- **Orient the agent quickly**
  - what this kit is
  - what the agent is building
  - what “done” means
- **Define the reading order**
  - what to read first, second, third
  - what to treat as highest authority if conflicts exist
- **Define execution behavior**
  - how to implement work (stepwise, small increments)
  - how to record progress and decisions
  - when to stop vs proceed
- **Define verification behavior**
  - when to run checks
  - what outputs count as proof

---

2) Instruction Content (what it must include)

- **Goal recap**
  - the agent’s mission in this build
- **Authority / precedence rules**
  - what sources override others if conflicting
- **Workflow**
  - the execution loop the agent must follow
  - how tasks are selected and sequenced
- **Tool use constraints**
  - what tools can be used
  - what is forbidden (e.g., inventing APIs, skipping checks)
- **Output discipline**
  - required format for progress updates (what it must report each cycle)

    - required state updates so the work can resume cleanly

---

3) Instruction Format Rules (to reduce misreads)

- **Single entrypoint** with minimal branching
- **Explicit “do not” rules** (hard constraints)
- **Short, structured sections** (easy for an agent to parse)
- **References by manifest/IDs** instead of vague “see the docs”
- **No narrative filler**—only operational rules

---

4) Failure Handling Rules (what agent must do when blocked)

The instruction layer must define what happens if:

- required info is missing
- the kit contains contradictions
- a check fails
- the runtime environment lacks a needed capability

Key requirement: the agent must not “smooth over” problems; it must surface them explicitly in a standard way.

---

5) Output Boundary

The Agent Instruction Layer is complete when an agent can:

- start from the entrypoint,
- understand what to build and how,
- navigate the kit correctly,
- execute in a controlled loop,
- and produce verifiable progress without improvising core decisions.

Execution Contracts

Execution Contracts

Definition

Execution Contracts are the **binding rules** that constrain how the agent builds so it remains accurate, consistent, and efficient. They exist to eliminate “creative interpretation” and enforce disciplined implementation.

---

1) Contract Types (what kinds of rules exist)

- **Behavior Contracts**
  - how the agent must operate (step size, reporting, decision handling)
- **Authority Contracts**
  - what sources of truth override others
  - how conflicts are resolved
- **Implementation Contracts**
  - how code must be structured
  - required conventions and patterns
  - constraints on dependencies and architecture
- **Safety & Integrity Contracts**
  - what must never be done (inventing requirements, bypassing security rules, silent changes)
- **Verification Contracts**
  - what must be proven and when
  - what commands/checks are required to claim completion

---

2) Core “Must / Never” Rules (the heart of anti-drift)

Execution Contracts should define non-negotiables such as:

Must

- follow the defined authority order
- implement only what is specified (or explicitly marked as unknown)
- keep work in small verifiable increments
- update state/progress in the required format
- run required checks and capture proof

Never

- invent missing requirements as if they were decided
- change standards silently
- skip verification and still claim completion
- create duplicate/conflicting sources of truth inside the project

---

3) Decision Handling (how the agent treats gaps)

Contracts must define:

- what counts as a “blocking unknown”
- what the agent is allowed to assume (if anything)
- how assumptions must be labeled and recorded
- when the agent must stop and surface the issue

---

4) Change Discipline

Execution Contracts should constrain how edits happen:

- where changes are allowed
- how refactors are handled
- how the agent prevents breaking unrelated areas
- how updates are documented so the build remains resumable

---

5) Proof Discipline

A contract must define what evidence is required:

- command outputs
- test results
- build success
- artifact presence (files updated/created)
- consistency checks (no broken references)

No proof = not complete.

---

6) Output Boundary

Execution Contracts are “complete” when:

- the agent has a clear set of enforceable rules,
- ambiguity is handled predictably,
- and the agent cannot claim progress without evidence.

Verification & Proof

Verification & Proof

Definition

Verification & Proof is the system that defines **how correctness is demonstrated**. It prevents "it should work" progress by requiring **evidence** that the build matches the plan and meets the quality bar.

---

1) Verification Objectives

- **Confirm correctness:** the product matches the specified intent
- **Confirm integrity:** no contradictions, broken references, or missing required pieces
- **Confirm stability:** changes don't silently break other parts
- **Confirm readiness:** the build can run and be validated in its environment

---

2) Verification Types (what kinds of checks exist)

- **Structural Checks**
  - required files exist
  - required sections are filled
  - manifests/indexes reference valid targets
  - no duplicate/conflicting "truth" artifacts
- **Consistency Checks**
  - definitions match usage
  - references resolve cleanly
  - permissions/roles align across places they appear
- **Build & Runtime Checks**
  - project builds successfully
  - app starts successfully
  - core flows can be exercised
- **Automated Test Checks**
  - unit tests (when applicable)
  - integration tests (when applicable)
  - regression tests for critical flows
- **Security & Policy Checks** (when applicable)
  - auth flows behave as required
  - access restrictions hold
  - sensitive data handling matches defined rules

---

3) Proof Artifacts (what counts as evidence)

Verification must specify acceptable proof formats, such as:

- command outputs (build/test/lint results)
- generated logs
- screenshots or recorded outputs (when UI proof is needed)
- structured checklists marked complete with references to outputs
- “before/after” diffs for critical changes

Key requirement: proof must be **capturable** and **auditable**, not just described.

---

4) Failure Handling Rules

Verification must define what happens when:

- a check fails
- evidence is missing
- the runtime environment can’t run a required check

The rule should be: failures are surfaced explicitly, not worked around silently.

---

5) Completion Criteria

A build is “complete” only when:

- all required verification checks pass, and
- required proof artifacts are present and linked to the checks they satisfy.

---

6) Output Boundary

Verification & Proof ends when:

- the system can point to concrete evidence that the product meets the spec and standards,
- and that evidence is packaged or recorded in a consistent place/format.

State & Continuity

State & Continuity

Definition

State & Continuity is how the system ensures work stays coherent across time—across pauses, tool sessions, platform switches, and even different agents—without losing accuracy or re-deciding key things.

---

1) What “state” includes

State is the **minimum information required to resume correctly** without reinterpreting intent.

It includes:

- **Progress status**
  - what is done, in progress, blocked, not started
- **Decisions**
  - resolved choices that should not be re-opened
  - any approved overrides to standards/rules
- **Open questions / unknowns**
  - unresolved items explicitly recorded
  - what is needed to resolve them
- **Change history (high-level)**
  - what changed recently and why (traceable)
- **Verification status**
  - what checks have passed/failed
  - what proof artifacts exist

---

2) Continuity guarantees (what it must enable)

A good state system enables:

- **Pause/resume** without rereading everything
- **Agent handoff** (new agent can continue without drift)
- **Platform portability** (works even if moved to another environment)
- **Conflict containment** (prevents old assumptions from resurfacing)

---

3) State update rules (when and how state changes)

State must be updated:

- after each completed work unit
- after any decision/override
- after any check run (pass/fail)
- when blocked (record what's blocking and why)

Key requirement: state updates are **structured and consistent**, not buried in narrative chat.

---

4) Resume rules (how to restart safely)

A resume process should be defined so the agent:

- reads the current state first
- aligns itself to the latest truth and progress
- continues from the next defined work unit
- does not repeat finished work unless a regression is detected

---

5) Drift prevention mechanisms inside state

State & Continuity reduces drift by:

- turning "memory" into explicit recorded facts
- freezing decisions once made (unless explicitly reopened)
- recording unknowns instead of letting the agent fill gaps silently
- tying verification proof to progress claims

---

6) Output boundary

State & Continuity is complete when:

- there is a single, clear, up-to-date record of "where we are,"
- it can be used to restart work reliably,
- and it prevents rework and reinterpretation across sessions.

Versioning & Evolution

Versioning & Evolution

Definition

Versioning & Evolution is how the system changes over time **without breaking stability**. It ensures outputs are reproducible, old kits remain usable, and improvements don't create "junkyard" fragmentation.

---

1) What must be versioned

To keep behavior consistent, versioning must apply to:

- **The system itself** (overall spec + behavior expectations)
- **Templates** (formats and required fields)
- **Standards library** (defaults, constraints, allowed options)
- **Pipeline logic** (how inputs become outputs)
- **Kit structure** (folder layout, manifests, entrypoints)

---

2) Version identifiers (what a version needs to communicate)

A version should communicate:

- **compatibility** (what it works with)
- **meaningful changes** (what behavior is different)
- **reproducibility** (what rules were applied)

At minimum, every kit should record:

- system version used to generate it
- standards version used
- template set version used

---

3) Compatibility rules (preventing breakage)

Versioning must define:

- what constitutes a breaking change vs non-breaking change
- whether older kits can be executed by newer agents/tools without modification

- how the system behaves when it encounters an older kit format

---

4) Migration rules (how change is applied safely)

When breaking changes happen, the system needs:

- a defined migration path (old format → new format)
- rules for automatic vs manual migration
- safeguards to prevent partial upgrades that cause contradictions

---

5) Deprecation rules (how old patterns die cleanly)

To avoid accumulating “junkyard parts,” define:

- how a template/standard is marked deprecated
- how long it remains supported
- what replaces it
- how kits are labeled when they use deprecated components

---

6) Change traceability (why something changed)

Every change should be traceable to:

- the problem it solves (drift, ambiguity, quality, portability)
- what component changed (template/standard/pipeline/structure)
- what outputs are affected

---

7) Output boundary

Versioning & Evolution is complete when:

- any kit can declare exactly what system produced it,
- updates don’t silently change outputs,
- and you can evolve the system without creating inconsistent parallel behaviors.

Governance & Control

Governance & Control

Definition

Governance & Control is the layer that keeps the system “manufactured,” not “junkyard.” It defines **who can change what**, **how changes are reviewed**, and **how the system stays internally consistent** as it grows.

---

1) Ownership Model (who controls what)

Governance defines clear ownership over:

- standards (design/engineering/security/quality rules)
- templates (structure, required fields, formatting)
- pipeline stages (transform rules, validations)
- kit structure (folder layout, manifests, entrypoints)
- verification rules (what proof is required)

Each area needs a single accountable owner (even if that’s just you today).

---

2) Change Admission Rules (what qualifies as an acceptable change)

A change should only be accepted if it:

- improves clarity, consistency, portability, or quality
- reduces drift or continuity errors
- has a defined scope and does not introduce duplicate truth
- includes an update to the relevant versioning and traceability

---

3) Review & Approval Flow (how changes get adopted)

Governance defines:

- when a change is “proposed”
- what review is required (self-review checklist or peer review)
- how acceptance is recorded
- how changes move from draft → active

The goal is to prevent silent, uncontrolled evolution.

---

4) Consistency Enforcement (stopping contradictions)

Controls should exist to prevent:

- overlapping responsibilities between templates
- conflicting standards
- multiple sources defining the same decision
- kits being generated with missing/invalid pieces

This is where “rules that block publication” live.

---

5) Release Discipline (how versions become official)

Governance defines:

- when a new version is released
- what testing/verification is required before release
- how compatibility is checked (old kits, older formats)
- rollback rules (what happens if a release is bad)

---

6) Auditability (why things exist)

Governance includes a system of record for:

- why a standard/template exists
- what problem it solved
- what it replaced
- what it impacts

This prevents “mystery parts” accumulating.

---

7) Output Boundary

Governance & Control is complete when:

- changes are intentional and traceable,

- the system cannot drift internally through unmanaged edits,
- and new additions don’t degrade clarity or stability.

Tooling & Platform Integration

Tooling & Platform Integration

Definition

Tooling & Platform Integration describes how the kit and the agent operate inside different build environments ("vibe-coding platforms") without breaking workflow, losing capabilities, or introducing environment-driven drift.

---

1) Environment Assumptions (what the system expects)

Define the minimum required capabilities the runtime must provide:

- file system read/write access
- ability to run commands (build/test/run)
- ability to install dependencies
- ability to view/edit project files reliably
- any network access requirements (if needed)

These assumptions must be explicit so the kit stays portable.

---

2) Platform Constraint Handling (what varies by platform)

Different environments may differ in:

- permission limits (restricted file access, blocked commands)
- terminal availability or behavior
- path/OS differences
- dependency install limitations
- sandboxing or network limits

The system must describe how the agent should adapt *without changing intent*.

---

3) Integration Interface (how the agent uses tools)

Define standard tool interaction patterns:

- where commands are run
- how outputs are captured as proof

- how files are edited (atomic edits vs broad rewrites)
- how the agent handles tool failures (retry vs stop)

---

4) Portability Rules (what keeps kits universal)

- avoid hard-coded absolute paths
- keep kit structure self-contained
- declare external dependencies instead of assuming them
- include manifests/indexes so navigation doesn't rely on platform UI

---

5) Minimum Viable "Adapter" Concept

If platforms vary significantly, define a small adapter layer:

- a standardized "how to run" section for each supported platform type
- environment-specific command variants if necessary
- a consistent method for locating entrypoints and state

---

6) Output Boundary

Tooling & Platform Integration is complete when:

- the kit can be imported/unzipped into a supported environment,
- the agent can execute the workflow using only declared capabilities,
- and environment differences do not cause reinterpretation or drift.

Artifacts

Below is a **complete parts list** framed as “LEGO pieces” — **physical artifacts** (files/objects) and **non-physical pieces** (rules/structures) that must exist for the system to work. Each piece has a purpose.

A) Physical Pieces (things you can point to)

A1) Intake Assets

1. **Intake Form**
    - Structured fields the user completes.
2. **Intake Schema**
    - The formal definition of valid form data (required/optional, types).
3. **Intake Submission Record**
    - The captured user responses in a stored format.
4. **Intake Validator**
    - Checks submission against the schema; flags missing/invalid data.

A2) Core Data Artifacts (system “truth objects”)

5. **Normalized Input Record**
    - Cleaned/standardized version of the submission.
6. **Resolved Standards Snapshot**
    - The final ruleset used for this project (version-stamped).
7. **Canonical Spec**
    - Single structured model of the product intent (IDs, relationships, unknowns).
8. **Work Breakdown**
    - Build units, dependencies, sequencing.
9. **Acceptance Map**
    - What “done” means per work unit (checks and expectations).
10. **State Snapshot**

- Current progress/decisions/unknowns/proof pointers (for continuity).

A3) Libraries (reusable stores)

11. **Template Library**

- The set of templates available to fill.

12. **Standards Library**

- Design/engineering/security/quality rules and defaults.

13. **Vocabulary/Definitions Library**

- Controlled language: terms, actions, tags/labels.

14. **Example/Reference Library** (optional)

- Canonical examples, patterns, “golden” references.

A4) Generators (things that produce artifacts)

15. **Normalizer**

- Converts raw intake → normalized input.

16. **Standards Resolver**

- Defaults + overrides → resolved standards snapshot.

17. **Spec Builder**

- Normalized input + standards → canonical spec.

18. **Planner / Decomposer**

- Canonical spec → work breakdown + acceptance map.

19. **Template Filler**

- Canonical spec + standards → filled document set.

20. **Packager**

- Collects outputs → structured kit → zip.

A5) Kit Deliverable (what gets handed to the agent)

21. **Agent Kit (Zip File)**

- The portable package.

22. **Kit Folder Structure**

- Standardized layout inside the zip.

23. **Entrypoint Document**

- Single “start here” file.

24. **Kit Manifest / Index**

- Map of what’s inside; where to find things; IDs.

25. **Project Standards Stamp**

- The versions and resolved ruleset included in the kit.

26. **Execution Checklist**

- Required checks + proof expectations.

A6) Verification Assets

27. **Verification Plan**

- What to test/check, when.

28. **Proof Log**

- Collected proof artifacts pointers (build output, test output, etc.).

29. **Validation Scripts / Commands List**
- The concrete checks the agent must run (or equivalents).

30. **Completion Report Format**
- Standardized “done” reporting format.

A7) Control Assets (stability over time)

31. **Version Registry**
- Tracks versions of templates/standards/system.

32. **Changelog**
- What changed and why.

33. **Compatibility Notes**
- Old kit formats vs new execution rules.

34. **Lock Rules**
- What is frozen / controlled vs flexible.

B) Non-Physical Pieces (rules, structures, invariants)

B1) Authority & Truth Rules

35. **Source-of-Truth Order**
- If conflicts exist, what wins.

36. **No-Duplicate-Truth Rule**
- Decisions defined once, referenced everywhere else.

37. **Unknowns Policy**
- How undecided items are represented (not guessed).

B2) Identity & Referencing System

38. **ID System**
- Stable IDs for spec items (features, rules, screens, data).

39. **Reference Rules**
- How items link to each other; what a valid reference is.

B3) Completeness & Quality Rules

40. **Minimum Completeness Criteria**
- What must exist before packaging is allowed.

41. **Quality Bar Rules**
- Standards that must be met to call something shippable.

B4) Execution Discipline Rules

42. **Agent Operating Loop**
- Required cycle: read → plan → act → verify → update state.

43. **Change Discipline**
- How edits are made; how scope changes are handled.

44. **Verification Discipline**
- No proof = not done.

B5) Packaging Invariants

45. **Stable Layout Contract**
- Kit structure cannot vary per project.

46. **Entrypoint Contract**
- Exactly one start file; predictable location.

47. **Manifest Completeness Rule**
- Everything in kit must be referenced by the manifest.

B6) Platform Portability Rules

48. **Environment Requirements Contract**
- Minimum capabilities needed to execute.

49. **Adapter Rules**
- How environment-specific differences are handled without changing intent.

C) “Atomic Pieces” inside the Canonical Spec (the smallest LEGO blocks)

These are the smallest *concept units* the system must be able to represent:

50. **User Role**

51. **Permission**
52. **Workflow**
53. **Capability / Feature**
54. **Screen / View**
55. **UI Action**
56. **System Operation**
57. **Data Object**
58. **Field**
59. **Relationship**
60. **Business Rule**
61. **Constraint**
62. **Acceptance Check**
63. **Test Case**
64. **External Integration**
65. **Nonfunctional Requirement** (performance, reliability, etc.)
66. **Unknown / Open Question**

That's the parts inventory.

If you want the next step, tell me whether you want to go **top-down** (A then B then C) or **bottom-up** (start with C, then build up).

Physical Artifacts

Physical Artifacts (detailed outline)

A1) Intake Assets

Purpose: capture intent and constraints in a structured way, and ensure it's valid before anything downstream.

1. **Intake Form**

- What it is: the required user-facing questionnaire.
- What it contains: structured fields for scope, users/roles, workflows, data, constraints, preferences, success criteria, references.
- Output: a submission payload.

2. **Intake Schema**

- What it is: the formal definition of valid submissions.
- Includes: required/optional fields, types, allowed values, length limits, field dependencies (if X then Y required).
- Output: validation rules used by the validator.

3. **Intake Submission Record**

- What it is: stored raw submission (as provided).
- Includes: timestamp, submitter, version of the form/schema used, raw answers.
- Output: immutable input record for traceability.

4. **Intake Validator**

- What it is: the checker that enforces the schema.
- Produces: validation report (pass/fail, missing fields, contradictions if detectable at this stage).
- Output: either "validated submission" or "rejected submission + errors."

A2) Core Data Artifacts (system truth objects)

Purpose: the concrete “working artifacts” the pipeline produces and the kit delivers.

5. **Normalized Input Record**

- What it is: cleaned/standardized version of the submission.
- Includes: normalized lists/enums, standardized naming, extracted structured items from free-text, resolved formatting.
- Output: canonicalized input for reliable downstream processing.

6. **Resolved Standards Snapshot**

- What it is: the exact standards/ruleset chosen for *this* project.
- Includes: defaults + overrides + versions + “fixed vs configurable” flags.
- Output: deterministic constraints that other artifacts must follow.

7. **Canonical Spec**

- What it is: the single structured truth model of the product.
- Includes: stable IDs, roles, workflows, capabilities, interfaces, operations, data model, rules, constraints, unknowns.
- Output: the definitive internal representation everything else references.

8. **Work Breakdown**

- What it is: the decomposition of the build into work units.
- Includes: unit IDs, dependencies, sequencing, scope per unit, mapping back to spec IDs.
- Output: “what to build, in what order.”

9. **Acceptance Map**

- What it is: definition of “done” per work unit.
- Includes: acceptance criteria, required checks, proof requirements, test coverage expectations.
- Output: measurable completion rules.

10. **State Snapshot**

- What it is: the resumable progress record.
- Includes: completed units, current unit, blockers, decisions, overrides, proofs collected, last known good verification status.
- Output: continuity anchor for the agent and for handoffs.

A3) Libraries (reusable stores)

Purpose: reusable building blocks that keep outputs consistent across projects.

11. **Template Library**

- Contains: all template definitions (structure + required fields + fill rules).
- Organized by: template type and use-case.
- Output: template set used by the filler.

12. **Standards Library**

- Contains: design/engineering/security/quality standards.
- Organized by: standard category + version.
- Output: inputs to the standards resolver.

13. **Vocabulary/Definitions Library**

- Contains: controlled language (terms, actions, tags/labels).
- Output: normalization rules + consistent phrasing/IDs.

14. **Example/Reference Library (optional)**

- Contains: canonical examples, reference patterns, sample outputs.
- Output: guide rails for generation and consistency.

A4) Generators (artifact producers)

Purpose: transforms that create the core artifacts.

15. **Normalizer**

- Input: validated submission record
- Output: normalized input record

16. **Standards Resolver**

- Input: normalized input + standards library (+ overrides)
- Output: resolved standards snapshot

17. **Spec Builder**

- Input: normalized input + resolved standards
- Output: canonical spec

18. **Planner / Decomposer**

- Input: canonical spec
- Output: work breakdown + acceptance map

19. **Template Filler**

- Input: canonical spec + resolved standards + template library
- Output: filled document set (the guidance docs)

20. **Packager**

- Input: all produced artifacts
- Output: structured kit folder + zipped kit

A5) Kit Deliverable (hand-off package)

Purpose: portable, predictable package the agent can execute from.

21. **Agent Kit (Zip File)**

- Contains: everything required to build + verify.
- Must include: entrypoint + manifests + specs + plans + standards snapshot + checks.

22. **Kit Folder Structure**

- Standard layout that never changes between projects.

23. **Entrypoint Document**

- Single “Start Here” artifact for the agent.

24. **Kit Manifest / Index**

- “Table of contents” with IDs → file paths; also lists required reading order.

25. **Project Standards Stamp**

- Versions + resolved rules recorded inside the kit.

26. **Execution Checklist**

- Explicit checklist of what must be done and proven.

A6) Verification Assets

Purpose: ensure the agent can produce proof, not claims.

27. **Verification Plan**

- Defines what must be checked and when.

28. **Proof Log**

- Where evidence is recorded/pointered (command outputs, screenshots, etc.).

29. **Validation Scripts / Commands List**

- The concrete commands/checks expected in the runtime.

30. **Completion Report Format**

- Standard “done” report template for agent output.

A7) Control Assets (stability over time)

Purpose: prevent the system from drifting as it evolves.

31. **Version Registry**

- Tracks versions of templates, standards, pipeline, kit format.

32. **Changelog**

- What changed, why, and impact.

33. **Compatibility Notes**

- How older kits/templates behave with newer rules.

34. **Lock Rules**

- What’s frozen vs flexible; what requires formal change control.

Intake Assets

Intake Assets

Intake Assets — Description

A1 is the **front door** of the system. These artifacts exist to **capture user intent in maximum detail** (through a required form) and convert it into a **valid, traceable input package** that the rest of the pipeline can trust.

The Intake Assets do four things end-to-end:

1. **Collect** structured project information in a consistent format (so projects don't start with vague, unstructured prompts).

2. **Enforce validity** by checking required fields, types, and dependencies (so downstream steps don't have to guess or patch missing foundations).

3. **Preserve traceability** by storing the raw submission exactly as provided (so the final kit can always be traced back to what the user actually said).

4. **Produce a clean handoff boundary**: a "validated submission" that becomes the official starting point for normalization and specification building.

In short: A1 ensures the system starts from **high-quality inputs**—because every downstream error, drift problem, or contradiction usually begins with weak intake.

A1) Intake Assets

Purpose: capture intent and constraints in a structured way, and ensure it's valid before anything downstream.

1. **Intake Form**

- What it is: the required user-facing questionnaire.

- What it contains: structured fields for scope, users/roles, workflows, data, constraints, preferences, success criteria, references.

- Output: a submission payload.

2. **Intake Schema**

- What it is: the formal definition of valid submissions.
- Includes: required/optional fields, types, allowed values, length limits, field dependencies (if X then Y required).
- Output: validation rules used by the validator.

3. **Intake Submission Record**

- What it is: stored raw submission (as provided).
- Includes: timestamp, submitter, version of the form/schema used, raw answers.
- Output: immutable input record for traceability.

4. **Intake Validator**

- What it is: the checker that enforces the schema.
- Produces: validation report (pass/fail, missing fields, contradictions if detectable at this stage).
- Output: either “validated submission” or “rejected submission + errors.”

Skill Level Rules (Depth Profile) — Intake Form Behavior

Definition: `skill_level` controls (a) which modules/pages are shown, (b) which fields are required, (c) how much scaffolding/help is presented, and (d) autofill drafting depth.

A) UI behavior by skill level

Beginner

- **Progressive disclosure:** show fewer sections per page; collapse advanced blocks behind “Show more”
- **More structure:** prefer dropdowns/chips/checklists over free text where possible
- **More guidance:** helper text + examples visible by default

- **Defaults:** preselect safe defaults when reasonable
- **Autofill:** strongly supported (still user-controlled), drafts more content

Intermediate

- Standard page layouts (what we designed)
- Balanced structured inputs + free-text
- Advanced blocks available but not forced unless gated
- Autofill drafts only high-confidence sections

Expert

- Full module set visible (less collapsing)
- Minimal helper text (optional “show help”)
- More advanced subforms enabled by default
- Higher validation thresholds (see below)
- Autofill drafts minimal; prioritizes accuracy

B) Requiredness thresholds (form validation rules)

These apply on top of “required field” rules:

Beginner minimums

- Must-haves: **≥ 1**
- Roles: **≥ 1**
- Workflows: **≥ 3**

- Out-of-scope: **≥ 1**
- Data objects: **only required if user indicates data is stored**, minimal detail allowed (name + purpose)

Intermediate minimums

- Must-haves: **≥ 3** (or ≥ 1 if project is explicitly tiny)
- Roles: **≥ 1**, with at least some capability intent
- Workflows: **≥ 3** with clear steps
- Out-of-scope: **≥ 2**
- Data objects: **≥ 2** when applicable, include key fields

Expert minimums

- Must-haves: **≥ 5** (or rank all if fewer)
- Roles: **≥ 2** when relevant (unless explicitly single-user)
- Workflows: **≥ 5** (or 3 core + 2 edge cases)
- Out-of-scope: **≥ 3**
- Data objects: deeper detail when applicable (fields + relationships)
- Verification/Done: must include specific must-pass checks (not just generic)

(If you want different numbers later, these are the knobs.)

C) Module visibility rules (depth-based)

- Beginner: advanced modules are **hidden by default** unless:
    - required by matrix preset, or

    - triggered by gates (payments, integrations, auth, etc.)
- Intermediate: standard modules for the preset are shown; advanced modules are optional unless gated
- Expert: advanced modules for the preset are shown by default

---

D) Autofill drafting rules

If autofill is enabled:

- Generated answers must be labeled **DRAFT**
- Must never overwrite user-entered content without explicit confirmation
- Must flag uncertainty instead of inventing facts
- Draft depth scales by skill level:
    - Beginner: broader drafting (still cautious)
    - Intermediate: moderate drafting
    - Expert: minimal drafting unless high-confidence

Form

Intake Form Layout (Page-by-Page) — Page 0: Routing

Purpose of Page 0: collect the routing profile that determines which modules/pages appear next.

Page 0 — Routing (Matrix Selector)

Section 0.1 — Skill Level

1. **Skill Level**
- **UI type:** Radio group (3 options)
- **Options:** Beginner / Intermediate / Expert
- **Required:** Yes

Section 0.2 — Build Class

2. **Project Category**
- **UI type:** Dropdown (single select)
- **Options:** Application / Service / Infrastructure / Library-SDK
- **Required:** Yes
- **Behavior:** Selecting a category filters the “Type Preset” list.
3. **Type Preset**
- **UI type:** Dropdown (single select) OR Card selector (recommended for UX)
- **Options:** (depends on chosen category; from the locked preset lists)
- **Required:** Yes
- **Behavior:** Selecting a type sets default module bundle + defaults for later fields.

Section 0.3 — Build Target

4. **Build Target**
- **UI type:** Radio group (2 options)
- **Options:** New build / Existing project enhancement
- **Required:** Yes
- **Behavior:** If “Existing,” later pages include repo/context/migration-specific sections.

Section 0.4 — Audience Context

5. **Audience Context**

- **UI type:** Radio group (3 options) with helper text
- **Options:** Consumer-facing / Internal-Enterprise / Developer-facing
- **Required:** Yes
- **Behavior:** Consumer-facing enables Ideology/Brand + Design Direction modules by default.

Section 0.5 — Autofill Choice

6. **Autofill Draft Answers**

- **UI type:** Toggle switch
- **Default:** Off
- **Required:** No
- **Behavior:** If On, after Page 1 submit, AI drafts Page 2+ fields marked as “DRAFT” (never overwrites user-entered text).

Section 0.6 — Continue

7. **Continue**

- **UI type:** Button (“Continue”)
- **Behavior:** Validates required fields; saves routing object.

Page 0 Output Object (stored)

- `skill_level`
- `category`
- `type_preset`
- `build_target`
- `audience_context`
- `autofill`

Intake Form Layout — Page 1: Project Basics + Inputs

Purpose of Page 1: capture the core project description + any source materials (zip/links/repo) that the pipeline can parse. This page produces the minimum payload needed to run **optional autofill**.

Page 1 — Project Basics + Inputs

Section 1.1 — Project Identity (required core)

1. **Project Name**

- **UI type:** Single-line text input
- **Placeholder:** “e.g., Kinnected Life”
- **Required:** Yes
- **Validation:** min length 2, max length (reasonable UI limit)

2. **Project Idea / Overview**

- **UI type:** Multi-line textarea (large)
- **Placeholder:** “Describe what you want to build in as much detail as possible…”
- **Required:** Yes
- **Validation:** min length (to discourage one-liners), max length

3. **One-liner** *(optional)*

- **UI type:** Single-line text input
- **Placeholder:** “e.g., A family timeline and legacy app”
- **Required:** No

4. **Core experience paragraph** *(optional)*

- **UI type:** Multi-line textarea (medium)

- **Prompt text:** “Describe the user experience from open → success in one paragraph.”
- **Required:** No

Section 1.2 — Source Materials (optional but high leverage)

5. **Upload Zip (source materials)**

- **UI type:** File upload (accept `.zip`)
- **Required:** No
- **Helper text:** “Upload architecture notes, docs, diagrams, specs, or starter code.”

6. **Reference Links**

- **UI type:** Repeatable list input (Add link)
- **Field type:** URL input + optional “label/name” text
- **Required:** No
- **Examples:** competitors, inspiration, docs, repo links, Figma, etc.

Section 1.3 — Existing Project Context (conditional)

Shown when **Build Target = Existing project enhancement** (from Page 0).

7. **Existing Repo Link**

- **UI type:** URL input
- **Required:** Yes (if existing project)
- **Placeholder:** “GitHub/Bitbucket/GitLab/Replit link…”

8. **Current State Summary**

- **UI type:** Multi-line textarea
- **Required:** Yes (if existing project)
- **Prompt:** “What exists today? What works? What’s broken?”

9. **What must NOT change?**

- **UI type:** Multi-line textarea or bullet-list builder
- **Required:** No (but strongly encouraged)
- **Examples:** routes, database schema, UI patterns, auth method, etc.

10. **Known Issues / Bugs**

- **UI type:** Repeatable list (title + description)
- **Required:** No

Section 1.4 — Delivery Preference (required light routing)

11. **Desired Scope**

- **UI type:** Radio group (3 options)
- **Options:** MVP / Full product / Unsure
- **Required:** Yes

12. **Priority Bias**

- **UI type:** Radio group (3 options)
- **Options:** Speed / Balanced / Quality
- **Required:** Yes

Section 1.5 — Continue + Autofill Trigger

13. **Next**

- **UI type:** Button (“Next”)
- **Behavior:**
    - Validates required fields.
    - Saves Page 1 payload.
    - If **Autofill = On**, triggers AI draft generation for Page 2+.

Page 1 Output Object (stored)

- `project_name`
- `project_overview`
- `project_oneliner` (optional)
- `core_experience_paragraph` (optional)
- `zip_upload` (optional file pointer)
- `reference_links[]` (optional)
- `existing_repo_link` (conditional)
- `current_state_summary` (conditional)
- `must_not_change` (optional)
- `known_issues[]` (optional)
- `desired_scope`

- `priority_bias`

Intake Form Layout — Page 2: Product Intent + Ideology/Brand

Purpose of Page 2: lock in the *meaning* of the product: problem, audience, goals, success, boundaries. If **Audience Context = Consumer-facing**, this page also captures ideology + brand identity inputs that will shape UX copy and product decisions.

Page 2 — Product Intent + Ideology/Brand

Section 2.1 — Problem Definition (required)

1. **What problem does this solve?**
- **UI type:** Multi-line textarea
- **Required:** Yes
2. **Why is this important now?**
- **UI type:** Multi-line textarea
- **Required:** No
3. **What do users do today instead? (alternatives / competitors / workarounds)**
- **UI type:** Repeatable list
  - Item fields: “Name” (text), “Link” (url optional), “What’s missing?” (textarea short)
- **Required:** No

Section 2.2 — Audience (required)

4. **Who is this for?**
- **UI type:** Multi-line textarea
- **Required:** Yes
5. **Primary user type (short label)**
- **UI type:** Single-line text input
- **Placeholder:** “e.g., independent artists” / “parents managing family memories”
- **Required:** Yes
6. **Secondary user types**
- **UI type:** Repeatable list (text input)
- **Required:** No

Section 2.3 — Goals & Success (required)

7. **Primary goals**
- **UI type:** Repeatable list (text input)
- **Required:** Yes (min 1)
8. **What does success look like?**
- **UI type:** Multi-line textarea
- **Required:** Yes
9. **Success metrics** *(optional but useful)*
- **UI type:** Repeatable list
    - Item fields: Metric name (text), Target (text/number), Timeframe (text optional)
- **Required:** No
10. **Non-goals / Out of scope**
- **UI type:** Repeatable list (text input) + optional “why out of scope” short text
- **Required:** Yes (min 1)

Consumer-Facing Only Blocks (shown when Audience Context = Consumer-facing)

Section 2.4 — Purpose & Belief (consumer-facing)

11. **What does this project/brand stand for?**
- **UI type:** Multi-line textarea
- **Required:** Yes (consumer-facing)
12. **What change should it create in the user’s life?**
- **UI type:** Multi-line textarea
- **Required:** No
13. **Core values**
- **UI type:** Multi-select chips + “Add custom value”
- **Default list examples:** (values list can be defined later)
- **Required:** No (recommended)
14. **Beliefs that shape product decisions**
- **UI type:** Multi-select chips + “Add custom belief”
- **Examples:** privacy-first, community-first, premium-first, accessibility-first
- **Required:** No
15. **What would you refuse to do even if it made money? (guardrails)**
- **UI type:** Multi-line textarea or list builder
- **Required:** No (recommended)

Section 2.5 — Audience Culture / Psychographics (consumer-facing)

16. **Who is the user as a person? (psychographics)**
- **UI type:** Multi-line textarea
- **Required:** No
17. **What do they care about most?**
- **UI type:** Multi-line textarea
- **Required:** No
18. **What do they fear/avoid?**
- **UI type:** Multi-line textarea
- **Required:** No
19. **Communities/cultures this aligns with**
- **UI type:** Repeatable list (text input)
- **Required:** No
20. **What "status" should the product signal?**
- **UI type:** Dropdown + optional custom text
- **Options (example):** Premium / Practical / Minimal / Bold / Rebellious / Luxury / Friendly / Professional / Other
- **Required:** No

Section 2.6 — Brand Positioning (consumer-facing)

21. **Brand promise (one sentence)**
- **UI type:** Single-line text input
- **Required:** Yes (consumer-facing)
22. **Positioning statement**
- **UI type:** Guided textarea with template hint:
    - "For ___ who ___, this is ___ that ___ unlike ___."
- **Required:** No
23. **Unique advantage**
- **UI type:** Multi-line textarea
- **Required:** No

Section 2.7 — Voice & Tone (consumer-facing)

24. **Voice adjectives**
- **UI type:** Multi-select chips + "Add custom"
- **Required:** Yes (consumer-facing)
25. **Tone boundaries (what it should never sound like)**

- **UI type:** Multi-line textarea
- **Required:** No

26. **Example phrases you want the UI to say**

- **UI type:** Repeatable list (text input)
- **Required:** No

27. **Content maturity**

- **UI type:** Radio group
- **Options:** Family-friendly / Mild language ok / Mature language ok
- **Required:** No

Section 2.8 — Emotional Journey (consumer-facing)

28. **How should users feel when they first open it?**

- **UI type:** Multi-line textarea
- **Required:** No

29. **How should they feel after completing the primary workflow?**

- **UI type:** Multi-line textarea
- **Required:** No

30. **What emotions should the UI avoid triggering?**

- **UI type:** Multi-line textarea
- **Required:** No

Section 2.9 — Continue

31. **Next**

- **UI type:** Button (“Next”)
- **Behavior:** saves Page 2 payload (and autofill drafts if enabled show as editable)

Page 2 Output Object (stored)

- Problem: `problem_statement`, `why_now`, `alternatives[]`
- Audience: `audience_description`, `primary_user_label`, `secondary_users[]`
- Goals: `primary_goals[]`, `success_definition`, `success_metrics[]`, `out_of_scope[]`
- Consumer-facing (conditional): ideology, psychographics, positioning, voice, emotional journey fields

---

Intake Form Layout — Page 3: Design Direction

Purpose of Page 3: give the agent explicit UI/UX direction so it doesn't "guess." This page is **default-required for Consumer-facing**, and **optional (but available)** for Internal/Enterprise and Developer-facing.

Page 3 — Design Direction

Section 3.1 — Visual Style Direction

1. **Style adjectives**
- **UI type:** Multi-select chips + "Add custom"
- **Required:** Yes if Consumer-facing; No otherwise
- **Helper text:** "Words like: premium, minimal, playful, bold, calm…"
2. **Visual direction preset**
- **UI type:** Dropdown (single select) + "Custom"
- **Required:** No
- **Options (example preset list):**
    - Minimal / Clean
    - Premium / Luxury
    - Bold / High-contrast
    - Friendly / Soft
    - Corporate / Professional
    - Retro / Nostalgic
    - Futuristic / Sci-fi
    - Editorial / Magazine
    - Other / Custom
3. **Avoid list (what you don't want it to look like)**
- **UI type:** Multi-line textarea or repeatable list
- **Required:** No

Section 3.2 — Brand System Inputs

4. **Logo upload**
- **UI type:** File upload (png/svg/jpg)
- **Required:** No
5. **Brand colors**
- **UI type:** Repeatable color picker + hex input

- **Fields per item:** Name (optional), Hex (required if item added)
- **Required:** No
- **Option:** “I don’t have colors yet” checkbox

6. **Fonts**

- **UI type:** Repeatable list
- **Fields per item:** Font name (text), Use case (Heading/Body/Mono optional)
- **Required:** No
- **Option:** “I don’t have fonts yet” checkbox

7. **Brand references**

- **UI type:** Repeatable list (URL + optional notes)
- **Required:** No
- **Prompt:** “Links to sites/brands whose look you like.”

Section 3.3 — Layout & UI Preferences

8. **UI density**

- **UI type:** Radio group
- **Options:** Compact / Balanced / Spacious
- **Required:** No

9. **Navigation preference**

- **UI type:** Dropdown (single select)
- **Options:** Bottom tabs / Sidebar / Top nav / Hybrid / Unsure
- **Required:** No

10. **Component vibe**

- **UI type:** Multi-select chips (or paired sliders)
- **Options examples:**
  - Rounded vs Sharp
  - Minimal vs Expressive
  - Flat vs Elevated
  - Simple vs Decorative
- **Required:** No

11. **Motion preference**

- **UI type:** Radio group
- **Options:** None / Subtle / Expressive
- **Required:** No

Section 3.4 — Accessibility & Content Presentation

12. **Accessibility expectations**

- **UI type:** Multi-select checklist + optional notes textarea
- **Options examples:**

    - High contrast
    - Large text support
    - Screen reader support
    - Keyboard navigation
    - Reduced motion
- **Required:** No

13. **Content presentation preference**

- **UI type:** Radio group
- **Options:** Text-heavy / Visual-first / Mixed / Unsure
- **Required:** No

Section 3.5 — Showcase Screens (high leverage)

14. **Screens that must look great**

- **UI type:** Repeatable list
- **Fields per item:**
    - Screen name (text)
    - What matters most visually (dropdown + optional notes)
        - Options: Clarity / Premium feel / Speed / Comfort / Fun / Trust / Other
    - Optional reference link/image (url or upload)
- **Required:** No (recommended for consumer-facing)

Section 3.6 — Continue

15. **Next**

- **UI type:** Button (“Next”)
- **Behavior:** saves Page 3 payload

Page 3 Output Object (stored)

- `style_adjectives[]`
- `visual_preset`
- `avoid_list`
- `logo_file`
- `brand_colors[]`
- `fonts[]`
- `brand_references[]`

- `ui_density`
- `nav_preference`
- `component_vibe[]` (or structured pairs)
- `motion_preference`
- `a11y_expectations[]` + `a11y_notes`
- `content_presentation`
- `showcase_screens[]`

Intake Form Layout — Page 4: Functional Spec (Features, Workflows, Roles, Rules)

Purpose of Page 4: capture the working blueprint of what the product must do: features, workflows, roles/permissions, and rule constraints. This is the main driver of build planning.

Page 4 — Functional Spec

Section 4.1 — Features (required)

1. **Core features (must-haves)**
- **UI type:** Repeatable list builder
    - Item fields: Feature name (text) + short description (optional)
- **Required:** Yes (min 1)
2. **Nice-to-have features**
- **UI type:** Repeatable list builder
- **Required:** No
3. **Future backlog**
- **UI type:** Repeatable list builder
- **Required:** No
4. **Priority ranking (Top 5 must-haves)**
- **UI type:** Drag-and-drop ranking list (from the must-haves)
- **Required:** Yes (if must-haves ≥ 5; otherwise rank all)

Section 4.2 — Users / Roles (required)

5. **Roles**

- **UI type:** Repeatable subform (accordion per role)
- **Required:** Yes (min 1 role)
- **Per-role fields:**
    - Role name (text, required)
    - Role description (textarea short, optional)
    - Primary goal (text, optional)

Section 4.3 — Permissions (required baseline)

6. **Permissions per role**

- **UI type:** Per-role permission builder (multi-select checklist + custom entries)
- **Required:** Yes (at least one capability per role)
- **Fields:**
    - Allowed capabilities (multi-select + “add custom”)
    - Restricted actions (optional multi-select/list)
    - Approval/step-up required actions (optional list)

Note: capabilities can be free-form here; later stages can normalize them into canonical operations.

Section 4.4 — Workflows (required)

7. **Top workflows (at least 3)**

- **UI type:** Repeatable workflow builder (accordion per workflow)
- **Required:** Yes (min 3)
- **Per-workflow fields:**
    - Workflow name (text, required)
    - Actor role (dropdown from roles, required)
    - Steps (ordered list; each step is a text input, required)
    - Success outcome (textarea short, required)
    - Failure states / what can go wrong (textarea short, optional)
    - Priority (dropdown: Critical / Important / Nice-to-have, optional)

8. **Edge-case workflows**

- **UI type:** Repeatable workflow builder (lighter version)
- **Required:** No

Section 4.5 — Business Rules & Validation (conditional)

9. **Are there business rules that must always be enforced?**

- **UI type:** Yes/No radio
- **Required:** No
- **Behavior:** If Yes, show rule fields below.

10. **Must-always rules**

- **UI type:** Repeatable list builder (text + optional examples)
- **Required:** Yes if business rules = Yes

11. **Must-never rules**

- **UI type:** Repeatable list builder
- **Required:** No

12. **Validation rules**

- **UI type:** Repeatable list builder
    - Field(s) affected (optional text)
    - Rule description (text, required)
- **Required:** No

13. **Lifecycle/state rules**

- **UI type:** Repeatable list builder
- **Required:** No
- **Example:** “Content goes Draft → Published → Archived.”

Section 4.6 — Continue

14. **Next**

- **UI type:** Button (“Next”)
- **Behavior:** saves Page 4 payload

Page 4 Output Object (stored)

- `must_have_features[]`
- `nice_to_have_features[]`
- `future_features[]`
- `feature_priority_rank[]`
- `roles[]` (role objects)
- `role_permissions[]` (by role)
- `workflows[]` (detailed)
- `edge_workflows[]` (optional)
- `business_rules_enabled`
- `must_always_rules[]`
- `must_never_rules[]`

- `validation_rules[]`
- `lifecycle_rules[]`

Intake Form Layout — Remaining Pages (Conditional, Full Superset)

Below is the **complete page layout** for everything after Page 4. Which of these pages appear is decided by the **matrix routing** + **gates** (auth needed, integrations, etc.). This layout includes **all fields from the inventory** and specifies **input types**.

Page 5 — Data Model & Content

Shows for: most Application/Service projects (and any time “stores/manages data” = yes).

5.1 Data Objects (required when enabled)

1. **Does the product store/manage data?**
- UI: Yes/No radio
- If No → skip page (or show minimal notes field)
2. **Core data objects**
- UI: Repeatable subform (accordion per object)
- Required: Yes if data is used (min 1)
- Per-object fields:
    - Object name (text, required)
    - Description/purpose (textarea short, optional)
    - Required fields (repeatable list: field name text + type dropdown + notes optional)
    - Optional fields (repeatable list: same structure)
    - Relationships (repeatable list)
        - Relationship type dropdown: has-many / belongs-to / one-to-one / references
        - Target object dropdown (from created objects) + notes optional
    - Lifecycle states (repeatable list, optional)

5.2 Sensitivity & Data Handling

3. **Sensitive data present?**
- UI: Multi-select checklist
- Options: PII / credentials / payments / precise location / minors / health / other

- Required: No
4. **Data import/export needs**
- UI: Multi-select + notes textarea
- Options: import / export / backups / migration / none
5. **Retention expectations**
- UI: Textarea
- Required: No

Page 6 — Auth & Authorization

Shows for: most apps/services; optional for infra/SDK unless relevant.

1. **Authentication required?**
- UI: Yes/No radio
- Required: Yes (so the system knows)

If Yes:
2) **Auth methods**

- UI: Multi-select checklist
- Options: Email+Password / Magic Link / OAuth / SSO / Passkeys / Other
- Required: Yes
3. **Account lifecycle**
- UI: Multi-select + notes
- Options: Signup / Invite-only / Admin-created / Password reset / Email verification
4. **2FA required?**
- UI: Yes/No radio
5. **Session rules**
- UI: Checklist + notes
- Options: multi-device sessions / remember me / session timeout / device management
6. **Authorization model**
- UI: Dropdown
- Options: Role-based / Attribute-based / Mixed / Unsure
7. **Approval flows needed?**
- UI: Yes/No + textarea (describe)

Page 7 — Integrations

Shows for: any project that needs third-party services.

1. **Any external integrations?**
- UI: Yes/No radio

If Yes:
2) **Integrations list**

- UI: Repeatable subform
- Per-integration fields:
    - Service name (text, required)
    - Purpose (textarea short, required)
    - Data in (textarea short, optional)
    - Data out (textarea short, optional)
    - Triggers (multi-select: webhook/event/scheduled/manual + notes)
    - Credentials/secrets handling expectations (textarea optional)

Page 8 — Non-Functional Requirements

Shows for: all, but requiredness increases with Intermediate/Expert.

1. **Performance targets**
- UI: Repeatable list (metric name + target + notes)
2. **Scale assumptions**
- UI: Numeric inputs + notes
    - Expected users (number)
    - Expected concurrent users (number)
    - Expected requests/traffic (text/number)
    - “Unsure” checkbox allowed
3. **Reliability expectations**
- UI: Dropdown + notes
- Options: Standard / High / Mission-critical / Unsure
4. **Offline requirements**
- UI: Yes/No + notes
5. **Compliance constraints**
- UI: Multi-select checklist + notes
- Options: Payments / Healthcare / Minors / GDPR-like privacy / SOC2-like controls / Other

---

Page 9A — Application UI Structure (Screens & Navigation)

Shows for: Application category types (UI-heavy).

1. **Primary screens/pages**
- UI: Repeatable subform (accordion per screen)
- Required: Yes (min 3 for most app presets)
- Per-screen fields:
    - Screen name (text, required)
    - Purpose (textarea short, required)
    - Roles who can access (multi-select from roles, required)
    - Data shown (textarea short, optional)
    - Actions available (repeatable list, required)
    - Empty/error states (textarea optional)
2. **Navigation structure**
- UI: Repeatable list or structured builder
- Fields:
    - Main nav sections (repeatable list)
    - Role-based differences (textarea optional)
3. **UX requirements**
- UI: Checklist + notes
- Options: onboarding / notifications / search / saved views / localization / accessibility-first

---

Page 9B — Service Structure (Operations & API Contract)

Shows for: Service category types.

1. **Core responsibilities**
- UI: Textarea (required)
2. **Core operations**
- UI: Repeatable list (operation name + description)
- Required: Yes (min 3)
3. **API consumers**
- UI: Repeatable list (consumer name + notes)
- Required: Yes
4. **Endpoints/actions**

- UI: Repeatable subform
- Per-endpoint fields:
    - Name/path (text)
    - Method/type (dropdown: GET/POST/etc. or “GraphQL op”)
    - Input summary (textarea)
    - Output summary (textarea)
    - Auth required (yes/no)
    - Rate limit needs (optional)

5. **Webhooks**

- UI: Repeatable list (event name + payload notes)

Page 9C — Infrastructure Structure (Ops/Platform)

Shows for: Infrastructure category types.

1. **Infrastructure goal**

- UI: Textarea (required)

2. **Environments**

- UI: Multi-select checklist
- Options: dev / stage / prod / other
- Required: Yes

3. **Deployment & runtime expectations**

- UI: Checklist + notes
- Options: CI/CD / blue-green deploys / infra-as-code / secrets mgmt / rollback

4. **Observability**

- UI: Checklist + notes
- Options: logs / metrics / tracing / alerts / dashboards

5. **Backups & recovery**

- UI: Textarea (optional)

Page 9D — Library/SDK Structure (DX)

Shows for: Library/SDK category types.

1. **Target language(s)**

- UI: Multi-select (JS/TS, Python, etc.) + other

- Required: Yes
2. **Developer persona**
- UI: Textarea (required)
3. **Core API surface**
- UI: Repeatable list
    - function/class name
    - purpose
    - inputs/outputs summary (optional)
4. **Initialization/config pattern**
- UI: Textarea (optional)
5. **Packaging/distribution**
- UI: Dropdown + notes (npm/pip/etc.)
6. **Docs/examples expectations**
- UI: Checklist + notes
- Options: quickstart / reference docs / code samples / tutorials

---

Page 10 — Verification & Done + Final Confirmation

Shows for: all projects (required).

10.1 Definition of Done

1. **Definition of done**
- UI: Multi-line textarea (required)
2. **Must-pass checks**
- UI: Multi-select checklist + “add custom”
- Examples: build passes / tests pass / lint passes / core workflows validated / security checks
3. **Acceptance criteria statements**
- UI: Repeatable list (optional but strong)
4. **Rejection conditions**
- UI: Multi-line textarea (optional)
- Prompt: “What would make you reject the build even if it mostly works?”

10.2 Final Confirmation

5. **Confirm priorities are correct**
- UI: Checkbox (required)
6. **Confirm out-of-scope is real**

- UI: Checkbox (required)

7. **Confirm constraints are accurate**

- UI: Checkbox (required)

8. **Submit**

- UI: Button (“Submit”)

Notes on Page Count

- You asked for 4 pages originally; this design supports **4 core pages + conditional pages**.
- In UI, you can present Pages 5–10 as:
    - separate pages (cleaner, less cognitive load), or
    - a stepper with collapsible sections.

Form Inventory

Updated Superset Intake Form Inventory (all form types)

Below is the **full universe** of intake pieces, updated to include the expanded **Ideology/Brand** capture and the new **Design Direction** page. Any specific form (Skill Level × Category/Type) is a subset.

Page 0 — Routing (Matrix Selector)

Module: Routing

- Skill Level (Beginner / Intermediate / Expert)
- Project Category (Application / Service / Infrastructure / Library-SDK)
- Project Type Preset (dependent on category)
- Consumer-facing? (Yes / No / Unsure) *(or inferred by preset)*
- Build target (New project / Existing project enhancement) *(optional)*
- Autofill Choice (Off / On)
- Continue

Output: `{skill_level, category, type_preset, consumer_facing, build_target, autofill}`

Page 1 — Project Basics + Inputs

Module: Project Identity

- Project Name *(required)*
- Project Idea / Overview *(required long text)*
- One-liner *(optional)*
- “Describe the core experience in one paragraph” *(optional)*

Module: Source Materials

- Upload Zip *(optional)*
- Links (docs, references, repos, inspo) *(optional list)*
- Existing repo? *(yes/no + link)*
- Existing architecture/spec? *(yes/no + upload/link)*

- Existing brand assets? *(yes/no + upload/link)*

Module: Output Preference

- Desired scope *(MVP / Full / Unsure)*
- Priority bias *(Speed / Quality / Balanced)*

Page 2 — Product Intent + Ideology + Brand (expanded)

2A) Problem Definition

Module: Problem

- What problem does this solve? *(required)*
- Why is this important now? *(optional)*
- Current alternatives/workarounds (competitors) *(optional)*

2B) Audience

Module: Audience

- Who is this for? *(required)*
- Primary user type description *(required)*
- Secondary user types *(optional)*

2C) Goals & Success

Module: Goals

- Primary goals *(required list)*
- What does success look like? *(required)*
- Success metrics *(optional list)*
- Non-goals / out-of-scope *(required list)*

2D) Purpose & Belief (consumer-facing module)

Module: Ideology

- What does this project/brand stand for? *(required if consumer-facing)*
- What change should it create in the user's life? *(optional)*
- Core values *(pick list + free text)*
- Product beliefs that shape decisions *(privacy-first, community-first, premium-first, etc.)*
- What would you refuse to do even if it made money? *(guardrails)*

2E) Audience Culture (consumer-facing module)

Module: Audience Psychographics

- Who is the user as a person? *(psychographic prompt)*
- What do they care about most?
- What do they fear/avoid?
- Communities/cultures this aligns with *(optional)*
- What "status" should the product signal? *(optional)*

2F) Brand Positioning (consumer-facing module)

Module: Positioning

- Brand promise (one sentence) *(required if consumer-facing)*
- Positioning statement template *(optional but strong)*
- 3 competitors/alternatives + differentiation *(optional)*
- Unique advantage *(optional)*

2G) Voice & Tone (consumer-facing module)

Module: Voice

- Voice adjectives *(required if consumer-facing)*
- Tone boundaries (what it should never sound like)
- Example phrases the UI should say *(optional)*
- Content maturity (family-friendly / mature language allowed / etc.) *(optional)*

2H) Emotional Journey (consumer-facing module)

Module: Emotional Journey

- How should users feel on first open?
- How should they feel after the primary workflow?
- What emotions should the UI avoid triggering?

Page 3 — Design Direction (new dedicated page)

3A) Visual Style

Module: Visual Direction

- Style adjectives *(required if consumer-facing; optional otherwise)*
- Pick a visual direction preset *(optional)* + “Custom”
- Avoid list *(optional)*

3B) Brand System Inputs

Module: Brand Assets

- Logo upload *(optional)*
- Colors (hex values or “none yet”) *(optional)*
- Fonts (names or “none yet”) *(optional)*
- Brand references (links/screenshots) *(optional)*

3C) Layout & UI Preferences

Module: UI Preferences

- Density (compact / balanced / spacious)
- Navigation preference (tabs/sidebar/bottom nav/etc.)
- Component vibe (rounded vs sharp, minimal vs expressive)
- Motion preference (none / subtle / expressive)

3D) Accessibility & Content Presentation

Module: A11y & Content

- Accessibility expectations *(optional)*
- Content style (text-heavy / visual-first / mixed)

3E) Example Screens (high leverage)

Module: Showcase Screens

- List 3–7 screens that must look great
- For each: what matters most visually (clarity/premium/speed/etc.)

Page 4 — Functional Spec (features, workflows, rules)

Module: Features

- Core features (must-haves) *(required list)*
- Nice-to-haves *(optional list)*
- Future backlog *(optional list)*
- Feature priority ranking *(required: rank top 5 must-haves)*

Module: Workflows

- Top workflows *(required; at least 3)*
    - actor (role)
    - steps
    - success outcome
    - failure states *(optional)*
- Edge-case workflows *(optional)*

Module: Business Rules

- Must-always rules *(optional list; required if indicated)*
- Must-never rules *(optional list)*
- Validation rules *(optional list)*
- Lifecycle/state rules *(optional)*

Module: Permissions

- Roles list *(required, at least 1)*
- For each role:
    - allowed capabilities *(required)*
    - restricted actions *(optional)*
    - approval/step-up requirements *(optional)*

Page 5+ — Category/Type Modules (conditionally included)

Application Modules

Module: Screens & Navigation

- Primary screens *(required for app types)*
- Per-screen: purpose, roles, data, actions, states
- Navigation structure + role differences *(optional)*

Module: UX Requirements

- Onboarding *(optional)*
- Notifications *(optional)*
- Localization *(optional)*

Service Modules

Module: Service Capabilities

- Responsibilities *(required)*
- Operations *(required)*
- Jobs/schedules *(optional)*
- Events/pubsub *(optional)*

Module: API Contract

- Consumers *(required)*
- Key endpoints/actions *(required)*
- Webhooks *(optional)*
- Rate limits *(optional)*

Infrastructure Modules

Module: Infrastructure Purpose

- What is being provisioned/managed *(required)*
- Environments *(required)*
- Scaling expectations *(optional)*

Module: Deployment & Runtime

- Hosting constraints *(optional)*
- CI/CD *(optional)*
- Observability *(optional)*
- Backup/recovery *(optional)*

Library/SDK Modules

Module: SDK Surface

- Target language(s) *(required)*
- Developer persona *(required)*
- Core API surface *(required list)*
- Config/init patterns *(optional)*

Module: Packaging & Distribution

- Packaging target *(optional)*
- Versioning expectations *(optional)*
- Examples/docs expectations *(optional)*

Cross-Cutting Modules (any category)

Module: Data Model

- Core data objects *(required for most projects)*
- Per-object: fields, relationships, ownership/tenancy *(optional)*
- Sensitive data flags *(optional)*
- Import/export *(optional)*
- Retention *(optional)*

Module: Auth

- Auth required? *(yes/no)*
- Auth method(s) *(if yes)*
- 2FA *(optional)*
- Sessions/devices *(optional)*
- Account lifecycle *(optional)*

Module: Integrations

- Integrations list *(optional)*
- For each: purpose, data in/out, triggers, secrets handling *(optional)*

Module: Non-Functional Requirements

- Performance *(optional)*
- Scale assumptions *(optional)*
- Reliability *(optional)*
- Offline *(optional)*
- Compliance *(optional)*

Module: Verification & Done

- Definition of done *(required)*
- Must-pass checks *(optional)*
- Acceptance criteria *(optional)*
- Rejection conditions *(optional)*

Module: Confirmation

- Confirm priorities
- Confirm out-of-scope
- Confirm constraints accuracy

Skill Level Depth Controls (Beginner / Intermediate / Expert)

Same inventory; changes requiredness + granularity.

- **Beginner:** fewer required subforms, more guided prompts
- **Intermediate:** structured roles/workflows + baseline data model + relevant category modules
- **Expert:** adds explicit constraints, detailed data model, verification expectations, integrations/NFRs where applicable

---

Autofill Touchpoints (draft-only)

Autofill may draft (marked “DRAFT”):

- Problem/Audience/Goals
- Ideology/Voice (if consumer-facing)

- Features/workflows/roles
- Initial screen list or operations list (category-dependent)
- Initial data objects

It must not overwrite user-entered answers without permission and must flag unknowns instead of inventing.

Form Matrix

Matrix Overview (Final — Locked)

What the matrix does

Selects **which intake modules/components appear** and **how deep/required** they are, based on the project's routing profile.

Locked Axes (4)

1) Skill Level (Depth Control)

- Beginner
- Intermediate
- Expert

2) Build Class (Relevance Control)

- **Category**: Application / Service / Infrastructure / Library-SDK
- **Type Preset**: selected within the chosen category (this is the primary driver of module bundle)

3) Build Target (Context Control)

- New build
- Existing project enhancement

4) Audience Context (Orientation Control)

- Consumer-facing
- Internal / Enterprise

- Developer-facing

Explicitly NOT Axes (handled as gates inside the form)

- MVP vs Full scope
- Compliance / high-stakes requirements (payments, minors, healthcare, etc.)
- Runtime surface (web/mobile/desktop/API-only) *(prefer encoded in Type Preset; otherwise a field)*
- Integrations depth, non-functional requirements depth, etc.

Routing Output (locked shape)

```
{ skill_level, category, type_preset, build_target, audience_context,
autofill }
```

Form Module Type Presets

Type Presets (Final — Locked)

Application

1. Consumer Mobile App
2. Consumer Web App
3. Cross-Platform App (Web + Mobile)
4. Internal Admin Dashboard
5. Marketplace App
6. Ecommerce Storefront
7. Social / Community App
8. Messaging / Chat App
9. Booking / Scheduling App
10. Content / Media App (feed, library, streaming)
11. Education / Course App
12. Fitness / Habit / Tracker App
13. Finance / Budgeting App (non-banking)
14. Event / Ticketing App
15. Portfolio / Landing / Marketing Site (app-like CMS optional)

Service

1. REST API Service
2. GraphQL API Service

3. Realtime Service (WebSocket/SSE)
4. Auth / Identity Service
5. Payments / Billing Service
6. Notifications Service (email/SMS/push)
7. Search Service
8. Media Processing Service (images/video/audio)
9. Data Ingestion / ETL Service
10. Workflow / Automation Service
11. Recommendation / Personalization Service
12. BFF (Backend-for-Frontend) Service
13. Integration Hub (connectors/webhooks)
14. Admin / Ops Service (audit, moderation, internal tools)

Infrastructure

1. CI/CD Pipeline Setup
2. Hosting + Deployment Stack (single app)
3. Multi-Environment Platform (dev/stage/prod)
4. Observability Stack (logs/metrics/tracing/alerts)
5. Database Infrastructure (Postgres + backups/replication)
6. Caching Layer (Redis or equivalent)
7. Message Queue / Event Bus Setup
8. File/Blob Storage Setup

9. Networking / Reverse Proxy / Gateway Setup
10. IAM / Secrets Management Setup
11. Data Warehouse / Analytics Stack
12. Reliability / DR (disaster recovery) Buildout
13. Security Hardening Baseline (policies, scanning, guardrails)

Library / SDK

1. Frontend Component Library (UI kit)
2. JS/TS SDK (API client)
3. Mobile SDK (iOS/Android wrapper)
4. CLI Tool
5. Backend Framework Plugin / Middleware
6. Data/Schema Validation Library
7. Auth SDK
8. Analytics / Telemetry SDK
9. Integration Connector Library
10. Testing Utilities Library

Schema

Intake Schema (Universal, Implementation-Agnostic)

1) Purpose / Function

The Intake Schema is the **formal contract** for what a valid intake submission is. It exists so the system can:

- validate user submissions deterministically (required/optional fields, types, allowed values, constraints)
- enforce **conditional requirements** (if X then Y required)
- produce a consistent validation output that downstream pipeline steps can trust

Output: validation rules used by the Intake Validator (pass/fail + violations).

---

2) Schema Shape (Top-Level Objects)

A submission is a single JSON-like object with these top-level sections:

1. `routing` *(required)* — matrix routing keys
2. `project` *(required)* — project identity text
3. `inputs` *(optional)* — attachments/links
4. `existing` *(conditional)* — required when enhancing an existing project
5. `delivery` *(required)* — scope & priority bias
6. `intent` *(required)* — problem, audience, goals, success, out-of-scope
7. `brand` *(conditional)* — consumer-facing ideology/voice fields

8. `design` *(conditional)* — consumer-facing minimum design direction + optional design prefs
9. `spec` *(required)* — features, roles, permissions, workflows, business rules
10. `data` *(optional gated)* — data model/content (enabled when applicable)
11. `auth` *(optional gated)* — auth/authorization details (enabled when applicable)
12. `integrations` *(optional gated)* — external integrations (enabled when applicable)
13. `nfr` *(optional)* — non-functional requirements
14. `app_ui` *(conditional)* — UI structure details when applicable
15. `service` *(conditional)* — service/API details when applicable
16. `infra` *(conditional)* — infra details when applicable
17. `sdk` *(conditional)* — SDK/library details when applicable
18. `final` *(required)* — definition of done + confirmations

Notes:

- "conditional/applicable" does not mean UI pages; it means **schema sections may be present** depending on what's being built and gate flags.

---

3) Field Definitions (Types + Requiredness + Constraints)

3.1 `routing` (required)

- `skill_level`: enum `SkillLevel` **required**
- `category`: enum `ProjectCategory` **required**
- `type_preset`: enum `TypePreset` **required**

- `build_target`: enum `BuildTarget` **required**
- `audience_context`: enum `AudienceContext` **required**
- `autofill`: boolean **required**

Constraint:

- `type_preset` must be valid for `category` (dependency rule).

3.2 `project` (required)

- `project_name`: string **required** (min/max length)
- `project_overview`: string **required** (min/max length)
- `project_oneliner`: string optional (max length)
- `core_experience_paragraph`: string optional (max length)

3.3 `inputs` (optional)

- `zip_upload`: fileRef optional (must be `.zip`, size limit)
- `reference_links[]`: optional array of `{ url: string, label?: string }` (url must be valid)

3.4 `existing` (conditional)

Required when `routing.build_target == existing`:

- `existing_repo_link`: url **required**

- `current_state_summary`: string **required** (min/max length)

Optional:

- `must_not_change[]`: array<string>
- `known_issues[]`: array<{ title: string, description?: string }>

3.5 `delivery` (required)

- `desired_scope`: enum `DesiredScope` **required**
- `priority_bias`: enum `PriorityBias` **required**

3.6 `intent` (required)

Required:

- `problem_statement`: string **required**
- `audience_description`: string **required**
- `primary_user_label`: string **required**
- `primary_goals[]`: array<string> **required** (min 1)
- `success_definition`: string **required**
- `out_of_scope[]`: array<string> **required** (min 1)

Optional:

- `why_now`: string

- `alternatives[]`: array<{ name: string, link?: url, whats_missing?: string }>
- `secondary_users[]`: array<string>
- `success_metrics[]`: array<{ metric: string, target?: string, timeframe?: string }>

3.7 `brand` (conditional: consumer-facing)

Required when `routing.audience_context == consumer`:

- `stands_for`: string **required**
- `brand_promise`: string **required**
- `voice_adjectives[]`: array<string> **required** (min 1)

Optional:

- `change_created`: string
- `core_values[]`: array<string>
- `decision_beliefs[]`: array<string>
- `refusal_guardrails`: string
- `psychographics`: string
- `user_cares_about`: string
- `user_fears_avoids`: string
- `aligned_communities[]`: array<string>
- `status_signal`: enum `StatusSignal`
- `positioning_statement`: string

- `unique_advantage`: string
- `tone_boundaries`: string
- `example_phrases[]`: array<string>
- `content_maturity`: enum `ContentMaturity`
- `first_open_emotion`: string
- `post_success_emotion`: string
- `avoid_emotions`: string

3.8 `design` (conditional: consumer-facing minimum)

Required when `routing.audience_context == consumer`:

- `style_adjectives[]`: array<string> **required** (min 1)

Optional:

- `visual_preset`: enum `VisualPreset`
- `avoid_list[]`: array<string>
- `logo_file`: fileRef (png/svg/jpg)
- `brand_colors[]`: array<{ name?: string, hex: string }> (hex format valid)
- `fonts[]`: array<{ name: string, use_case?: enum FontUseCase }>
- `brand_references[]`: array<{ url: string, notes?: string }> (valid url)
- `ui_density`: enum `UIDensity`
- `nav_preference`: enum `NavPreference`

- `component_vibe[]`: array<string>
- `motion_preference`: enum `MotionPreference`
- `a11y_expectations[]`: array<string>
- `a11y_notes`: string
- `content_presentation`: enum `ContentPresentation`
- `showcase_screens[]`: array<{ name: string, visual_priority?: enum ShowcasePriority, notes?: string, ref_url?: url, ref_file?: fileRef }>

3.9 `spec` (required)

Required:

- `must_have_features[]`: array<{ name: string, description?: string }> **required** (min 1)
- `feature_priority_rank[]`: array<string> **required** (must reference must-have feature names; length rule)
- `roles[]`: array<{ name: string, description?: string, primary_goal?: string }> **required** (min 1; names unique)
- `role_permissions[]`: array<{ role_name: string, allowed_capabilities[]: array<string>, restricted_actions[]?: array<string>, approval_required_actions[]?: array<string> }> **required**
    - each `role_name` must exist in roles
    - `allowed_capabilities` min 1
- `workflows[]`: array<{ name: string, actor_role: string, steps[]: array<string>, success_outcome: string, failure_states?: string, priority?: enum WorkflowPriority }> **required** (min 3)

    - `actor_role` must exist in roles
    - steps min 1

Optional:

- `nice_to_have_features[]`: same shape
- `future_features[]`: same shape
- `edge_workflows[]`: same shape as workflows
- `business_rules_enabled`: boolean
- `must_always_rules[]`: array<string> **required if enabled**
- `must_never_rules[]`: array<string>
- `validation_rules[]`: array<{ fields?: string, rule: string }>
- `lifecycle_rules[]`: array<string>

3.10 `data` (optional gated)

Gate:

- `enabled`: boolean

If `data.enabled == true`:

- `objects[]`: array of:
  `{ name: string, description?: string, required_fields[]?: array<{name,type,notes?}>, optional_fields[]?: same, relationships[]?: array<{relation_type,target_object,notes?}>, lifecycle_states[]?: array<string> }`
- Constraints:

    - object names unique
    - `relationships.target_object` references an object name
      Optional:

- `sensitive_data_flags[]`: enum list `SensitiveDataFlag`
- `import_export_needs[]`: enum list `ImportExportNeed`
- `retention_expectations`: string
- `ownership_tenancy_notes`: string

3.11 `auth` (optional gated)

Gate:

- `required`: boolean

If `auth.required == true`:

- `methods[]`: enum list `AuthMethod` **required** (min 1)
  Optional:
- `account_lifecycle[]`: enum list `AccountLifecycle`
- `two_fa_required`: boolean
- `session_rules[]`: enum list `SessionRule`
- `authorization_model`: enum `AuthorizationModel`
- `approval_flows_needed`: boolean
- `approval_flows_description`: string **required if approval_flows_needed==true**

3.12 `integrations` (optional gated)

Gate:

- `enabled`: boolean

If `integrations.enabled == true`:

- `items[]`: array<{ service_name: string, purpose: string, data_in?: string, data_out?: string, triggers[]?: enum list IntegrationTrigger, secrets_handling?: string }> **required** (min 1)

3.13 `nfr` (optional)

- `performance_targets[]`: array<{ metric: string, target: string, notes?: string }>
- `scale_assumptions`: { expected_users?: number, concurrent_users?: number, traffic_notes?: string, unsure?: boolean }
- `reliability_expectation`: enum `ReliabilityLevel`
- `offline_required`: boolean
- `compliance_flags[]`: enum list `ComplianceFlag`

3.14 Category-specific sections (conditional, when applicable)

These sections are allowed and may become required depending on project type; schema defines shape:

`app_ui`

- `screens[]`: array<{ name, purpose, roles_allowed[]: string, data_shown?, actions[]: string, empty_error_states? }>

- `navigation`: { main_sections[]?, role_differences? }
- `ux_requirements[]`: enum list `UXRequirement`
  Constraints: `roles_allowed` references `spec.roles[].name`

`service`

- `responsibilities`: string
- `operations[]`: array<{ name, description? }>
- `api_consumers[]`: array<{ name, notes? }>
- `endpoints[]`: array<{ name_or_path, method_type, input_summary?, output_summary?, auth_required?, rate_limit_notes? }>
- `webhooks[]`: array<{ event_name, payload_notes? }>

`infra`

- `goal`: string
- `environments[]`: enum list `Environment`
- `runtime_expectations[]`: enum list `RuntimeExpectation`
- `observability_needs[]`: enum list `ObservabilityNeed`
- `backups_recovery_notes`: string

`sdk`

- `target_languages[]`: array<string>
- `developer_persona`: string
- `api_surface[]`: array<{ name, purpose?, io_summary? }>

- `init_config_pattern`: string
- `packaging_distribution`: string
- `docs_examples_expectations[]`: enum list `DocsExpectation`

3.15 `final` (required)

- `definition_of_done`: string **required** (min/max length)
- `must_pass_checks[]`: array<string> optional
- `acceptance_criteria[]`: array<string> optional
- `rejection_conditions`: string optional
- `confirm_priorities`: boolean **required and must be true**
- `confirm_out_of_scope`: boolean **required and must be true**
- `confirm_constraints`: boolean **required and must be true**

---

4) Allowed Values (Enum List)

Enums used by the schema:

- `SkillLevel`, `ProjectCategory`, `TypePreset`, `BuildTarget`, `AudienceContext`
- `DesiredScope`, `PriorityBias`
- `StatusSignal`, `ContentMaturity`
- `VisualPreset`, `FontUseCase`, `UIDensity`, `NavPreference`, `MotionPreference`, `ContentPresentation`, `ShowcasePriority`

- `WorkflowPriority`
- `FieldType`, `RelationshipType`, `SensitiveDataFlag`, `ImportExportNeed`
- `AuthMethod`, `AccountLifecycle`, `SessionRule`, `AuthorizationModel`
- `IntegrationTrigger`
- `ReliabilityLevel`, `ComplianceFlag`
- `UXRequirement`, `Environment`, `RuntimeExpectation`, `ObservabilityNeed`, `DocsExpectation`

(Exact allowed values are the ones we already listed earlier; this schema references them.)

---

5) Dependencies (If X → Then Y)

Core dependency families:

1. `routing.category` determines valid `routing.type_preset`
2. If `routing.build_target == existing` → require `existing.existing_repo_link`, `existing.current_state_summary`
3. If `routing.audience_context == consumer` → require `brand.stands_for`, `brand.brand_promise`, `brand.voice_adjectives[]`, and `design.style_adjectives[]`
4. If `spec.business_rules_enabled == true` → require `spec.must_always_rules[]` (min 1)
5. If `data.enabled == true` → require `data.objects[]` (min 1)
6. If `auth.required == true` → require `auth.methods[]` (min 1)

7. If `auth.approval_flows_needed == true` → require `auth.approval_flows_description`

8. If `integrations.enabled == true` → require `integrations.items[]` (min 1)

9. Reference integrity:

    - `spec.role_permissions.role_name` must exist in `spec.roles.name`

    - `spec.workflows.actor_role` must exist in `spec.roles.name`

    - `app_ui.screens.roles_allowed[]` must exist in `spec.roles.name`

    - `data.objects.relationships.target_object` must exist in `data.objects.name`

    - `spec.feature_priority_rank[]` must reference `spec.must_have_features.name`

10. Format integrity:

- URLs valid

- Zip file is zip

- Hex colors valid

11. Uniqueness:

- role names unique

- data object names unique

---

6) Validation Output (Validator Contract)

Validator returns:

- `is_valid`: boolean
- `errors[]`: list of issues with:
  - `field_path`
  - `error_code`
  - optional `message`
  - `severity` (error)
  - optional `meta`

Optional warnings support is allowed, but not required.

Error codes (catalog):

- REQUIRED
- DEPENDENCY_MISSING
- MIN_ITEMS
- INVALID_ENUM
- INVALID_ENUM_FOR_CONTEXT
- INVALID_FORMAT
- INVALID_URL
- INVALID_FILETYPE
- INVALID_REFERENCE
- DUPLICATE_VALUE
- WARNING_INCOMPLETE *(optional)*

---

7) Skill-Level Minimum Thresholds (Schema-level rules)

These are schema validation rules keyed by `routing.skill_level`:

- Beginner:
  - `spec.must_have_features >= 1`
  - `spec.workflows >= 3`
  - `intent.out_of_scope >= 1`
  - `spec.roles >= 1`
- Intermediate:
  - `spec.must_have_features >= 3`
  - `spec.workflows >= 3`
  - `intent.out_of_scope >= 2`
  - `spec.roles >= 1`
- Expert:
  - `spec.must_have_features >= 5`
  - `intent.out_of_scope >= 3`
  - Workflows: `spec.workflows >= 5` OR (`spec.workflows >= 3` AND `spec.edge_workflows >= 2`)
  - Roles: target `>= 2` when relevant (or keep >=1 if you don't want the conditional)

Conditional thresholds (only when gates are enabled / sections applicable):

- If `data.enabled == true`: enforce higher depth for Intermediate/Expert (object counts, field depth)

- Expert may require at least one NFR signal (if you choose strictness)

Schema Blueprint

Universal Intake Schema Blueprint — Full Spec Grouped by Page (Option 2)

This is the **full spec grouped page-by-page**, with:

- **Field Registry entries** (field IDs, data type, UI input type, requiredness, constraints, allowed values)
- **Page-level dependencies**
- **Page-level profile thresholds** *(global thresholds listed once at the end, but referenced where relevant)*
- **Enums used on the page**

Page 0 — Routing (Matrix Selector)

Fields

- `routing.skill_level`
  - Type: enum `SkillLevel`
  - UI: radio
  - Required: yes
- `routing.category`
  - Type: enum `ProjectCategory`
  - UI: dropdown
  - Required: yes

- `routing.type_preset`
  - Type: enum `TypePreset`
  - UI: dropdown/cards
  - Required: yes
  - Constraint: must be valid for selected category
- `routing.build_target`
  - Type: enum `BuildTarget`
  - UI: radio
  - Required: yes
- `routing.audience_context`
  - Type: enum `AudienceContext`
  - UI: radio
  - Required: yes
- `routing.autofill`
  - Type: boolean
  - UI: toggle
  - Required: yes (default false)

Dependencies

- If `routing.category == application` → `routing.type_preset` ∈ Application presets (`INVALID_ENUM_FOR_CONTEXT`)

- If `routing.category == service` → `routing.type_preset` ∈ Service presets (`INVALID_ENUM_FOR_CONTEXT`)
- If `routing.category == infrastructure` → `routing.type_preset` ∈ Infrastructure presets (`INVALID_ENUM_FOR_CONTEXT`)
- If `routing.category == library_sdk` → `routing.type_preset` ∈ Library/SDK presets (`INVALID_ENUM_FOR_CONTEXT`)

Enums

- `SkillLevel`: beginner | intermediate | expert
- `ProjectCategory`: application | service | infrastructure | library_sdk
- `BuildTarget`: new | existing
- `AudienceContext`: consumer | internal_enterprise | developer
- `TypePreset`: full superset (category-scoped validity enforced)

Page 1 — Project Basics + Inputs

Fields

Project Identity

- `project.project_name`
    - Type: string
    - UI: text
    - Required: yes

    - Constraints: min length, max length
- `project.project_overview`
    - Type: string
    - UI: textarea (large)
    - Required: yes
    - Constraints: min length, max length
- `project.project_oneliner`
    - Type: string
    - UI: text
    - Required: no
    - Constraints: max length
- `project.core_experience_paragraph`
    - Type: string
    - UI: textarea (medium)
    - Required: no
    - Constraints: max length

Source Materials

- `inputs.zip_upload`
    - Type: fileRef
    - UI: file upload (.zip)
    - Required: no

    - Constraints: must be .zip; size limit

- `inputs.reference_links[]`
    - Type: array<object> `{ url: string, label?: string }`
    - UI: repeatable URL + optional label
    - Required: no
    - Constraints: each url must be valid

Existing Project Context (conditional)

- `existing.existing_repo_link`
    - Type: url
    - UI: URL input
    - Required: **yes if** `routing.build_target == existing`
- `existing.current_state_summary`
    - Type: string
    - UI: textarea
    - Required: **yes if** `routing.build_target == existing`
    - Constraints: min length, max length
- `existing.must_not_change[]`
    - Type: array<string>
    - UI: list builder
    - Required: no

- `existing.known_issues[]`
    - Type: array<object> `{ title: string, description?: string }`
    - UI: repeatable (title + optional description)
    - Required: no
    - Constraint: title required

Delivery Preference

- `delivery.desired_scope`
    - Type: enum `DesiredScope`
    - UI: radio
    - Required: yes
- `delivery.priority_bias`
    - Type: enum `PriorityBias`
    - UI: radio
    - Required: yes

Dependencies

- If `routing.build_target == existing` → require `existing.existing_repo_link`, `existing.current_state_summary` (`DEPENDENCY_MISSING`)
- If `inputs.zip_upload` present → must be `.zip` (`INVALID_FILETYPE`)
- If `inputs.reference_links` has items → each url valid (`INVALID_URL`)

Enums

- `DesiredScope`: mvp | full | unsure
- `PriorityBias`: speed | balanced | quality

Page 2 — Product Intent + Ideology/Brand

Fields

Problem Definition

- `intent.problem_statement`
  - Type: string
  - UI: textarea
  - Required: yes
  - Constraints: min/max length
- `intent.why_now`
  - Type: string
  - UI: textarea
  - Required: no
- `intent.alternatives[]`
  - Type: array<object> `{ name: string, link?: url, whats_missing?: string }`
  - UI: repeatable list

    - Required: no
    - Constraints: name required; link valid if present

Audience

- `intent.audience_description`
    - Type: string
    - UI: textarea
    - Required: yes
- `intent.primary_user_label`
    - Type: string
    - UI: text
    - Required: yes
- `intent.secondary_users[]`
    - Type: array<string>
    - UI: repeatable list
    - Required: no

Goals & Success

- `intent.primary_goals[]`
    - Type: array<string>
    - UI: repeatable list
    - Required: yes (min 1)

- `intent.success_definition`
  - Type: string
  - UI: textarea
  - Required: yes
- `intent.success_metrics[]`
  - Type: array<object> `{ metric: string, target?: string, timeframe?: string }`
  - UI: repeatable list
  - Required: no
  - Constraints: metric required
- `intent.out_of_scope[]`
  - Type: array<string>
  - UI: repeatable list
  - Required: yes (min 1)

Consumer-facing only (when `routing.audience_context == consumer`)

Purpose & Belief

- `brand.stands_for` (string, textarea) **required if consumer**
- `brand.change_created` (string, textarea) optional
- `brand.core_values[]` (array<string>, chips) optional
- `brand.decision_beliefs[]` (array<string>, chips) optional
- `brand.refusal_guardrails` (string, textarea) optional

Audience Psychographics

- `brand.psychographics` (string, textarea) optional
- `brand.user_cares_about` (string, textarea) optional
- `brand.user_fears_avoids` (string, textarea) optional
- `brand.aligned_communities[]` (array<string>, list) optional
- `brand.status_signal` (enum `StatusSignal`, dropdown) optional

Positioning

- `brand.brand_promise` (string, text) **required if consumer**
- `brand.positioning_statement` (string, guided textarea) optional
- `brand.unique_advantage` (string, textarea) optional

Voice & Tone

- `brand.voice_adjectives[]` (array<string>, chips) **required if consumer (min 1)**
- `brand.tone_boundaries` (string, textarea) optional
- `brand.example_phrases[]` (array<string>, list) optional
- `brand.content_maturity` (enum `ContentMaturity`, radio) optional

Emotional Journey

- `brand.first_open_emotion` (string, textarea) optional
- `brand.post_success_emotion` (string, textarea) optional
- `brand.avoid_emotions` (string, textarea) optional

Dependencies

- If `routing.audience_context == consumer` → require:
    - `brand.stands_for`, `brand.brand_promise`, `brand.voice_adjectives` (`DEPENDENCY_MISSING`)
- If `intent.alternatives[].link` present → URL valid (`INVALID_URL`)

Enums

- `StatusSignal`: premium | practical | minimal | bold | rebellious | luxury | friendly | professional | other
- `ContentMaturity`: family_friendly | mild_ok | mature_ok

Page 3 — Design Direction

Fields

Visual Style

- `design.style_adjectives[]`
    - Type: array<string>
    - UI: chips
    - Required: **yes if** consumer (min 1)
- `design.visual_preset`
    - Type: enum `VisualPreset`

    - UI: dropdown
    - Required: no
- `design.avoid_list[]`
    - Type: array<string>
    - UI: list builder
    - Required: no

Brand Assets

- `design.logo_file`
    - Type: fileRef
    - UI: upload (png/svg/jpg)
    - Required: no
    - Constraints: allowed filetypes; size limit
- `design.brand_colors[]`
    - Type: array<object> `{ name?: string, hex: string }`
    - UI: color picker + hex
    - Required: no
    - Constraints: hex valid (`INVALID_FORMAT`)
- `design.fonts[]`
    - Type: array<object> `{ name: string, use_case?: FontUseCase }`
    - UI: repeatable list

    - Required: no
    - Constraints: name required when item exists
- `design.brand_references[]`
    - Type: array<object> `{ url: string, notes?: string }`
    - UI: repeatable URL + notes
    - Required: no
    - Constraints: url valid

UI Preferences

- `design.ui_density` (enum `UIDensity`, radio) optional
- `design.nav_preference` (enum `NavPreference`, dropdown) optional
- `design.component_vibe[]` (array<string>, chips) optional
- `design.motion_preference` (enum `MotionPreference`, radio) optional

Accessibility & Content

- `design.a11y_expectations[]` (array<string>, checklist) optional
- `design.a11y_notes` (string, textarea) optional
- `design.content_presentation` (enum `ContentPresentation`, radio) optional

Showcase Screens

- `design.showcase_screens[]`
    - Type: array<object> `{ name: string, visual_priority?: ShowcasePriority, notes?: string, ref_url?: url, ref_file?:`

`fileRef }`

- UI: repeatable subform
- Required: no
- Constraints: name required; ref_url valid if present

Dependencies

- If consumer → require `design.style_adjectives.length >= 1` (`DEPENDENCY_MISSING` or `MIN_ITEMS`)
- Brand reference URLs valid (`INVALID_URL`)
- Showcase `ref_url` valid if present (`INVALID_URL`)
- Color hex valid (`INVALID_FORMAT`)

Enums

- `VisualPreset`: minimal_clean | premium_luxury | bold_high_contrast | friendly_soft | corporate_professional | retro_nostalgic | futuristic_scifi | editorial_magazine | other_custom
- `FontUseCase`: heading | body | mono | other
- `UIDensity`: compact | balanced | spacious
- `NavPreference`: bottom_tabs | sidebar | top_nav | hybrid | unsure
- `MotionPreference`: none | subtle | expressive
- `ContentPresentation`: text_heavy | visual_first | mixed | unsure
- `ShowcasePriority`: clarity | premium_feel | speed | comfort | fun | trust | other

---

Page 4 — Functional Spec

Fields

Features

- `spec.must_have_features[]`
    - Type: array<object> `{ name: string, description?: string }`
    - UI: repeatable list
    - Required: yes (min 1)
- `spec.nice_to_have_features[]` optional (same shape)
- `spec.future_features[]` optional (same shape)
- `spec.feature_priority_rank[]`
    - Type: array<string>
    - UI: ranking list
    - Required: yes
    - Constraint: entries must reference `spec.must_have_features[].name`

Roles

- `spec.roles[]`
    - Type: array<object> `{ name: string, description?: string, primary_goal?: string }`
    - UI: repeatable subform
    - Required: yes (min 1)

    - Constraint: role names unique

Permissions

- `spec.role_permissions[]`
    - Type: array<object> `{ role_name: string, allowed_capabilities: array<string>, restricted_actions?: array<string>, approval_required_actions?: array<string> }`
    - UI: per-role builder
    - Required: yes
    - Constraints: role_name references roles; allowed_capabilities min 1

Workflows

- `spec.workflows[]`
    - Type: array<object> `{ name: string, actor_role: string, steps: array<string>, success_outcome: string, failure_states?: string, priority?: WorkflowPriority }`
    - UI: workflow builder
    - Required: yes (min 3)
    - Constraints: actor_role references roles; steps min 1
- `spec.edge_workflows[]` optional (same shape)

Business Rules

- `spec.business_rules_enabled` (boolean, yes/no) optional
- `spec.must_always_rules[]` (array<string>) **required if enabled**

- `spec.must_never_rules[]` optional
- `spec.validation_rules[]` optional (`{ fields?: string, rule: string }`)
- `spec.lifecycle_rules[]` optional

Dependencies

- Priority rank entries exist in must-haves (`INVALID_REFERENCE`)
- Role names unique (`DUPLICATE_VALUE`)
- Permissions role_name exists (`INVALID_REFERENCE`)
- Workflow actor_role exists (`INVALID_REFERENCE`)
- If business_rules_enabled → must_always_rules min 1 (`DEPENDENCY_MISSING`)

Enums

- `WorkflowPriority`: critical | important | nice_to_have

Page 5 — Data Model & Content (conditional)

Gate Field

- `data.enabled` (boolean, yes/no)

Fields (when enabled)

- `data.objects[]` (array<object>) required if enabled
    - object shape: `{ name, description?, required_fields?, optional_fields?, relationships?, lifecycle_states? }`
    - `required_fields[]` / `optional_fields[]`: `{ name: string, type: FieldType, notes?: string }`
    - `relationships[]`: `{ relation_type: RelationshipType, target_object: string, notes?: string }`
    - `lifecycle_states[]`: array<string>
- `data.sensitive_data_flags[]` (enum list) optional
- `data.import_export_needs[]` (enum list) optional
- `data.retention_expectations` (textarea) optional
- `data.ownership_tenancy_notes` (textarea) optional

Dependencies

- If data.enabled → require data.objects min 1 (`DEPENDENCY_MISSING` / `MIN_ITEMS`)
- Object names unique (`DUPLICATE_VALUE`)
- relationships.target_object references an object name (`INVALID_REFERENCE`)

Enums

- `FieldType`: string | number | boolean | date_time | enum | json | file_ref | url | uuid | other
- `RelationshipType`: has_many | belongs_to | one_to_one | references

- `SensitiveDataFlag`: pii | credentials | payments | precise_location | minors | health | other
- `ImportExportNeed`: import | export | backups | migration | none

Page 6 — Auth & Authorization (conditional)

Gate Field

- `auth.required` (boolean)

Fields (when auth.required=true)

- `auth.methods[]` (enum list) required (min 1)
- `auth.account_lifecycle[]` (enum list) optional
- `auth.two_fa_required` (boolean) optional
- `auth.session_rules[]` (enum list) optional
- `auth.authorization_model` (enum) optional
- `auth.approval_flows_needed` (boolean) optional
- `auth.approval_flows_description` (textarea) required if approval_flows_needed=true

Dependencies

- If auth.required → require auth.methods (`DEPENDENCY_MISSING` / `MIN_ITEMS`)

- If approval_flows_needed → require approval_flows_description (`DEPENDENCY_MISSING`)

Enums

- `AuthMethod`: email_password | magic_link | oauth | sso | passkeys | other
- `AccountLifecycle`: signup | invite_only | admin_created | password_reset | email_verification
- `SessionRule`: multi_device_sessions | remember_me | session_timeout | device_management
- `AuthorizationModel`: role_based | attribute_based | mixed | unsure

Page 7 — Integrations (conditional)

Gate Field

- `integrations.enabled` (boolean)

Fields (when enabled)

- `integrations.items[]` (array<object>) required (min 1)
    - `{ service_name: string, purpose: string, data_in?: string, data_out?: string, triggers?: IntegrationTrigger[], secrets_handling?: string }`

Dependencies

- If integrations.enabled → require integrations.items min 1 (`DEPENDENCY_MISSING` / `MIN_ITEMS`)

Enums

- `IntegrationTrigger`: webhook | event | scheduled | manual | other

Page 8 — Non-Functional Requirements

Fields

- `nfr.performance_targets[]` (array<object> `{ metric, target, notes? }`) optional
- `nfr.scale_assumptions` (object `{ expected_users?, concurrent_users?, traffic_notes?, unsure? }`) optional
- `nfr.reliability_expectation` (enum ReliabilityLevel) optional
- `nfr.offline_required` (boolean) optional
- `nfr.compliance_flags[]` (enum list ComplianceFlag) optional

Enums

- `ReliabilityLevel`: standard | high | mission_critical | unsure
- `ComplianceFlag`: payments | healthcare | minors | privacy_gdpr_like | controls_soc2_like | other

---

Page 9A — Application UI Structure (conditional: category=application)

Fields

- `app_ui.screens[]` (array<object>) *(required when included)*
    - `{ name: string, purpose: string, roles_allowed: string[], data_shown?: string, actions: string[], empty_error_states?: string }`
- `app_ui.navigation` (object `{ main_sections?: string[], role_differences?: string }`) optional
- `app_ui.ux_requirements[]` (enum list UXRequirement) optional

Dependencies

- `roles_allowed[]` references `spec.roles[].name` (`INVALID_REFERENCE`)

Enums

- `UXRequirement`: onboarding | notifications | search | saved_views | localization | accessibility_first | other

---

Page 9B — Service Structure (conditional: category=service)

Fields

- `service.responsibilities` (textarea) required when included
- `service.operations[]` (array<object> `{ name, description? }`) requiredness by preset/profile
- `service.api_consumers[]` (array<object> `{ name, notes? }`) requiredness by preset/profile
- `service.endpoints[]` (array<object> `{ name_or_path, method_type, input_summary?, output_summary?, auth_required?, rate_limit_notes? }`) optional/conditional
- `service.webhooks[]` (array<object> `{ event_name, payload_notes? }`) optional

Page 9C — Infrastructure Structure (conditional: category=infrastructure)

Fields

- `infra.goal` (textarea) required when included
- `infra.environments[]` (enum list Environment) required when included
- `infra.runtime_expectations[]` (enum list RuntimeExpectation) optional
- `infra.observability_needs[]` (enum list ObservabilityNeed) optional
- `infra.backups_recovery_notes` (textarea) optional

Enums

- `Environment`: dev | stage | prod | other

- `RuntimeExpectation`: ci_cd | blue_green_deploys | infra_as_code | secrets_management | rollback | other
- `ObservabilityNeed`: logs | metrics | tracing | alerts | dashboards | other

---

Page 9D — Library/SDK Structure (conditional: category=library_sdk)

Fields

- `sdk.target_languages[]` (array<string>) required when included
- `sdk.developer_persona` (textarea) required when included
- `sdk.api_surface[]` (array<object> `{ name, purpose?, io_summary? }`) required when included
- `sdk.init_config_pattern` (textarea) optional
- `sdk.packaging_distribution` (string) optional
- `sdk.docs_examples_expectations[]` (enum list DocsExpectation) optional

Enums

- `DocsExpectation`: quickstart | reference_docs | code_samples | tutorials | other

---

Page 10 — Final Verification + Confirmation

Fields

- `final.definition_of_done` (textarea) required
- `final.must_pass_checks[]` (array<string>) optional
- `final.acceptance_criteria[]` (array<string>) optional
- `final.rejection_conditions` (textarea) optional
- `final.confirm_priorities` (checkbox boolean) required (must be true)
- `final.confirm_out_of_scope` (checkbox boolean) required (must be true)
- `final.confirm_constraints` (checkbox boolean) required (must be true)

Dependencies

- Confirmations must be true (`REQUIRED`)

Global Profile Threshold Rules (Section 5 reference)

Apply across pages:

- Beginner / Intermediate / Expert minimums for:
    - `spec.must_have_features` count
    - `spec.workflows` count (+ expert edge-workflow alternative)
    - `intent.out_of_scope` count
    - `spec.roles` count (expert higher)

    - Consumer: brand + design minimums
    - Existing: repo + current state minimums
    - If data.enabled: data object depth
    - Category pages: screens/ops depth (if enforced)

Global Validation Error Model (Section 6 reference)

- Payload shape: `ValidationIssue`
- Codes: REQUIRED, DEPENDENCY_MISSING, MIN_ITEMS, INVALID_ENUM, INVALID_ENUM_FOR_CONTEXT, INVALID_FORMAT, INVALID_URL, INVALID_FILETYPE, INVALID_REFERENCE, DUPLICATE_VALUE, WARNING_INCOMPLETE (optional)

Submission Record — Artifact Outline

Submission Record — Artifact Outline (end-to-end)

1) Artifact Name

Intake Submission Record (immutable raw intake snapshot)

2) Function

Store the user's intake submission **exactly as provided**, with enough metadata to guarantee:

- traceability (what the user actually said)
- reproducibility (which form/schema rules applied)
- auditability (when/who/what inputs were used)

This record is the **source input** for validation + normalization, but the record itself is not modified.

---

3) Inputs

- Raw form payload (all fields user submitted)
- Uploaded artifacts metadata (zip upload pointer, logo file pointer, etc.)
- System metadata captured at submission time (versions, timestamps)

---

4) Outputs

- A single **immutable record** representing the submission
- A stable `submission_id` reference usable by downstream steps

---

5) Record Structure (canonical fields)

5.1 Identity / Metadata

- `submission_id` (unique ID)
- `created_at` (timestamp)
- `created_by` (submitter identifier)
    - can be user ID, email, account ID, or “anonymous” depending on your auth model
- `source` (where it came from)
    - e.g., web app, internal tool, API
- `status` (optional)
    - e.g., received / validated / rejected *(status changes can be stored separately; the raw payload remains immutable)*

5.2 Version Stamps (critical for traceability)

- `form_version` (which UI form version produced this)
- `schema_version` (which Intake Schema version applies)
- `preset_snapshot` (optional but recommended)
    - the exact routing selection at the time: `{ skill_level, category, type_preset, build_target, audience_context, autofill }`

5.3 Raw Payload (immutable)

- `raw_submission` (the raw answers exactly as provided)
    - stored as the full submission object (same shape as the form outputs)
    - no normalization, no rewriting, no inference

5.4 Files / Attachments References (immutable pointers)

- `uploads[]` (list of file references)
    - each entry includes:
        - `file_id` or storage pointer
        - `file_name`
        - `file_type`
        - `file_size`
        - `field_path` (which field it came from, e.g., `inputs.zip_upload`, `design.logo_file`)
        - `uploaded_at`

5.5 Integrity / Audit (optional but strong)

- `payload_hash` (hash of raw_submission + upload refs)
- `ip_address` (optional, if captured)
- `user_agent` (optional)
- `notes` (optional)

---

6) Invariants (must always be true)

- The record is **append-only / immutable**:
    - raw fields are never edited after creation

- Any derived/processed forms (validated submission, normalized input) are stored as **separate artifacts** linked by `submission_id`

- Version stamps must always be present (`form_version`, `schema_version`)

---

7) Failure Modes / Edge Cases

- Partial uploads (file upload started but failed)

- User resubmits multiple times (each gets a new submission_id)

- Schema version changes between draft and submit (record must stamp versions at submit time)

- Missing user identity (anonymous) — still record source + timestamp

---

8) Interfaces (who uses it)

- **Intake Validator** reads `raw_submission` + version stamps

- **Normalizer** consumes the record only after validation (or consumes with validation output attached)

- **Audit/trace**: any later artifact can reference `submission_id` to prove provenance

Validator

Intake Validator — Artifact Outline (end-to-end)

1) Artifact Name

Intake Validator (submission validation engine)

2) Function

Validate an **Intake Submission Record** against the **Intake Schema**, producing a deterministic validation outcome:

- **PASS:** submission is valid and can proceed to normalization
- **FAIL:** submission is invalid; return structured errors (and optional warnings)

The validator is the **gatekeeper** that prevents drift downstream by refusing malformed, incomplete, or contradictory inputs.

---

3) Inputs

1. **Intake Submission Record**
- `submission_id`
- `form_version`
- `schema_version`
- `raw_submission`
- upload references (file refs, url lists)
2. **Intake Schema**
- field definitions (required/optional, types)
- enums
- constraints (min/max lengths, min items)
- dependency rules (if X then Y required)
- profile thresholds (Beginner/Intermediate/Expert)
- error model (error codes + payload shape)

---

4) Outputs

4.1 Validation Result

- `submission_id`

- `is_valid` (boolean)
- `errors[]` (list of validation issues)
- `warnings[]` (optional)
- `validated_at` (timestamp)
- `schema_version_used`
- `form_version_used`

4.2 Validation Issue (per error/warning)

- `field_path`
- `error_code`
- `message` (optional)
- `severity` (error|warning)
- `meta` (optional; expected vs actual, allowed values, etc.)

---

5) Validation Stages (order matters)

Stage 1 — Structural / Required Field Validation

- Check presence of required top-level sections and required fields
- Enforce required “must be true” confirmations

Fail produces: `REQUIRED`

Stage 2 — Type & Format Validation

- Validate field types (string/boolean/array/object)
- Validate formats (URL, hex color, file type)
- Validate enum membership

Fail produces: `INVALID_ENUM`, `INVALID_FORMAT`, `INVALID_URL`, `INVALID_FILETYPE`

Stage 3 — Dependency Validation

Enforce “if X then Y required” rules such as:

- build_target existing → require existing repo + current state

- consumer-facing → require brand minimums + design minimums
- business_rules_enabled → require must_always_rules
- auth.required → require auth.methods
- integrations.enabled → require integrations.items

Fail produces: `DEPENDENCY_MISSING`

Stage 4 — Referential Integrity & Uniqueness

Enforce cross-field consistency:

- role names unique
- workflows.actor_role exists in roles
- permissions.role_name exists in roles
- feature_priority_rank references must-have feature names
- data relationships reference existing data objects

Fail produces: `INVALID_REFERENCE`, `DUPLICATE_VALUE`

Stage 5 — Profile Threshold Validation (Skill Level)

Apply Beginner/Intermediate/Expert minimum depth rules:

- must-have count minimums
- workflows minimums (+ expert edge-workflow alternative)
- out-of-scope minimums
- any conditional thresholds (data depth if enabled, etc.)

Fail produces: `MIN_ITEMS` (or `WARNING_INCOMPLETE` if you treat some as warnings)

---

6) Determinism & Invariants

- Same submission + same schema version ⇒ same validation result
- Validator must never “fix” the submission; it only reports violations
- All errors must reference stable `field_path` keys from the schema
- Validation must record which versions were used

---

7) Failure Modes / Edge Cases

- Schema mismatch (submission references fields no longer in schema version)
  - policy: ignore unknown fields or flag them (choose one; common is warning)
- Missing uploads referenced in payload (file ref missing)
  - error: `DEPENDENCY_MISSING` or `INVALID_REFERENCE`
- Optional sections present but malformed
  - must be validated if present (no “partial bypass”)

---

8) Interfaces (what consumes the output)

- **Normalizer** consumes only submissions where `is_valid == true`
- **UI** consumes `errors[]` to show field-specific messages
- **Audit/tracing** links validation output to `submission_id` and versions

---

A1 Completion Check

A1 Intake Assets is now complete:

- Intake Form ✅
- Intake Schema ✅
- Intake Submission Record ✅
- Intake Validator ✅

Core Data Artifacts

Core Data Artifacts (system truth objects)

Purpose: the concrete “working artifacts” the pipeline produces and the kit delivers.

5. **Normalized Input Record**

- What it is: cleaned/standardized version of the submission.
- Includes: normalized lists/enums, standardized naming, extracted structured items from free-text, resolved formatting.
- Output: canonicalized input for reliable downstream processing.

6. **Resolved Standards Snapshot**

- What it is: the exact standards/ruleset chosen for *this* project.
- Includes: defaults + overrides + versions + “fixed vs configurable” flags.
- Output: deterministic constraints that other artifacts must follow.

7. **Canonical Spec**

- What it is: the single structured truth model of the product.
- Includes: stable IDs, roles, workflows, capabilities, interfaces, operations, data model, rules, constraints, unknowns.
- Output: the definitive internal representation everything else references.

8. **Work Breakdown**

- What it is: the decomposition of the build into work units.
- Includes: unit IDs, dependencies, sequencing, scope per unit, mapping back to spec IDs.
- Output: “what to build, in what order.”

9. **Acceptance Map**

- What it is: definition of “done” per work unit.

- Includes: acceptance criteria, required checks, proof requirements, test coverage expectations.

- Output: measurable completion rules.

10. **State Snapshot**

- What it is: the resumable progress record.

- Includes: completed units, current unit, blockers, decisions, overrides, proofs collected, last known good verification status.

- Output: continuity anchor for the agent and for handoffs.

Normalized Input Record

Normalized Input Record (Artifact Outline)

1) Artifact Name

Normalized Input Record (canonicalized intake payload)

2) Function

Convert a **validated Intake Submission Record** into a clean, standardized, machine-friendly input object that downstream steps can consume reliably without re-parsing or guessing.

This is the bridge between “raw user answers” and “structured system truth.”

---

3) Inputs

- **Validated Intake Submission Record**
  - `submission_id`
  - `raw_submission` (unchanged raw answers)
  - upload refs (zip/logo/etc.)
- **Intake Schema**
  - field paths, types, enums
  - dependency outcomes (what was required)
- **Validation Result**
  - confirms `is_valid == true`
  - warnings (optional, informative)

---

4) Outputs

A single **Normalized Input Record** linked to the submission:

- `submission_id` (foreign key / link)
- `normalized_input_id` (unique ID)
- `normalized_payload` (canonicalized object)
- `normalization_report` (what changed + what was inferred vs left unknown)
- `created_at`
- `schema_version_used`
- `form_version_used`

---

5) Normalization Rules (what “cleaned/standardized” means)

5.1 Type normalization

- Ensure all fields match schema types (strings trimmed, arrays always arrays, booleans normalized).
- Convert single values to arrays where schema expects arrays (if allowed by form UI).

5.2 Enum normalization

- Convert display labels to canonical enum values (e.g., “Cross-Platform App” → `cross_platform_app_web_mobile`).

5.3 Whitespace & formatting cleanup

- Trim whitespace, normalize newlines.
- Remove accidental duplicate list items (case/whitespace-insensitive duplicates).

5.4 Canonical naming normalization

- Normalize names used as references:
  - role names
  - feature names
  - workflow names
  - data object names
- Apply consistent casing/spacing rules (e.g., collapse multiple spaces).
- Preserve original text elsewhere (do not rewrite user intent).

5.5 Structured extraction (from free text)

Only when safe and explicitly part of normalization:

- Extract bullet lists from textareas into arrays **if** user clearly wrote list-form content.
- Extract URLs into link lists if pasted into free text.

Rule: extraction must be reversible/auditable in the normalization report.

5.6 Unknown/unclear handling

- If input is ambiguous, record as explicit `unknown` fields rather than filling guesses.
- Carry forward validator warnings but do not “resolve” them.

---

6) Normalization Report (traceability requirement)

The record should include a structured report like:

- `changes[]`:

    - `field_path`
    - `change_type` (trimmed, deduped, label_to_enum, extracted_list, normalized_name, etc.)
    - `before` (short/hashed or preview)
    - `after` (short/hashed or preview)
- `unknowns[]` (field paths that remain unclear)
- `notes` (optional)

---

7) Invariants

- Must preserve a direct link to the raw submission (`submission_id`).
- Must not introduce new requirements or “invent” content.
- Must be deterministic given the same submission + schema version + normalization ruleset version.

---

8) Failure Modes / Edge Cases

- User uses same label twice with slightly different spelling (dedupe policy must be consistent)
- Role names differ only by case (“Admin” vs “admin”) (must unify or flag)
- Free text contains mixed content (don’t over-extract; leave as text if uncertain)
- Uploaded zip exists but cannot be parsed (store pointer; parsing is not normalization unless you explicitly add it later)

---

9) Interfaces

- **Standards Resolver** consumes normalized routing + constraints inputs
- **Spec Builder** consumes normalized intent/spec inputs
- **Planner/Decomposer** and later stages rely on canonical names and arrays being stable

Resolved Standards Snapshot (Artifact Outline)

Resolved Standards Snapshot (Artifact Outline)

1) Artifact Name

Resolved Standards Snapshot (project-specific resolved ruleset)

2) Function

Produce the **exact standards/ruleset** that will govern this project build—resolved from:

- global defaults
- preset defaults (from category/type)
- any user-specified overrides (if allowed)

This snapshot exists so downstream artifacts (spec, plan, templates) can be **deterministic and consistent**.

---

3) Inputs

- **Normalized Input Record**
  - routing (`skill_level`, `category`, `type_preset`, `audience_context`, `build_target`)
  - any explicit preferences/constraints captured (design direction, auth preferences, etc.)
- **Standards Library** (global)
  - design standards
  - engineering standards
  - security standards
  - quality/verification standards

- **Version Registry**
    - which version of standards library is active/selected

---

4) Outputs

A single **Resolved Standards Snapshot** linked to the project:

- `resolved_standards_id`
- `submission_id` (link)
- `standards_version_used`
- `resolved_ruleset` (the actual final standards)
- `overrides_applied[]` (what was overridden and why)
- `fixed_vs_configurable` map (what is locked vs selectable)
- `created_at`

---

5) Snapshot Contents (what it must capture)

5.1 Identity / Versioning

- Standards library version
- Any preset pack versions (if you separate by category/type)
- Date/time generated

5.2 Design Standards (if applicable)

- typography rules (font choices or selection rules)

- spacing/layout rules
- component styling rules (buttons/forms/cards)
- accessibility minimums
- brand/voice rules (if consumer-facing)

5.3 Engineering Standards

- stack decisions (languages/frameworks/tooling)
- repo/project structure conventions
- coding conventions (naming, formatting, error handling)
- dependency policies (allowed/disallowed libs)

5.4 Security Standards

- authentication pattern requirements
- authorization enforcement expectations
- data handling requirements (PII, logging constraints)
- rate limiting/abuse controls (if applicable)

5.5 Quality / Verification Standards

- required checks (build/lint/tests)
- minimum acceptance expectations
- proof artifact expectations (what outputs count as proof)

5.6 Operational Standards (if applicable)

- environments (dev/stage/prod expectations)

- logging/monitoring expectations

---

6) Resolution Rules (how standards are chosen)

Order of resolution (deterministic):

1. **Global defaults**
2. **Category defaults**
3. **Type preset defaults**
4. **Audience context defaults** (consumer/internal/developer)
5. **Skill level adjustments** (requiredness thresholds, not “style changes”)
6. **User overrides** (only for fields marked configurable)

Rule: if an override conflicts with a fixed standard, it is rejected or recorded as blocked (policy choice).

---

7) Overrides Applied (traceability)

Every override entry should record:

- `field_path` (what was overridden)
- `before` value
- `after` value
- `source` (user input / preset / admin config)
- `reason` (optional)
- `status` (applied / blocked)

---

8) Invariants

- Same normalized input + same standards library version ⇒ same snapshot.
- Snapshot must be self-contained: downstream artifacts should not need to re-load “current standards.”
- Snapshot must explicitly label fixed vs configurable rules.

---

9) Failure Modes / Edge Cases

- Missing standard for a required domain (must fail loudly or fall back to explicit default)
- Conflicting presets (category/type/audience mismatch)
- User overrides without permission (must be blocked, not silently applied)
- Standards library version drift mid-project (snapshot locks it)

---

10) Interfaces

- **Canonical Spec builder** uses standards to constrain decisions (auth pattern, design direction, etc.)
- **Template filling** uses snapshot as authoritative “how to write/build”
- **Verification expectations** flow into Acceptance Map and DoD sections

Canonical Spec

A2.3 — Canonical Spec (Artifact Outline)

1) Artifact Name

Canonical Spec (single structured truth model for the project)

2) Function

Convert the normalized input + resolved standards into a **single, internally consistent, referenceable spec** that represents "what is being built" in a way that:

- prevents duplicate truths
- supports planning/decomposition
- supports deterministic template filling
- supports continuity across agents/sessions

This is the **authoritative product model** for the kit.

---

3) Inputs

- **Normalized Input Record**
  - intent (problem/audience/goals/features/workflows/roles/etc.)
  - conditional sections (data/auth/integrations/etc.) if present
- **Resolved Standards Snapshot**
  - constraints and defaults that must be reflected (stack, auth requirements, design rules, quality rules)
- **Vocabulary/Definitions Library** *(if used)*
  - canonical terms/actions/tags (optional)

---

4) Outputs

A single **Canonical Spec** linked to the project:

- `spec_id`
- `submission_id`
- `standards_version_used`
- `spec_version` (internal version for this project)
- `spec_graph` / `spec_document` (structured representation)
- `spec_index` (lookup tables for IDs)

- `created_at`

---

5) Canonical Spec Contents (what it must represent)

5.1 Identity + Context

- project identity (name/one-liner)
- routing snapshot (skill/category/type/build_target/audience)
- goals/success/out-of-scope

5.2 Roles & Permissions

- roles (stable IDs)
- permissions/capabilities (stable IDs)
- authorization notes (as constraints, not implementation)

5.3 Capabilities & Workflows

- capabilities/features (stable IDs)
- prioritized features (rank)
- workflows (stable IDs) mapped to:
    - actor role
    - steps
    - success outcomes
    - failure cases (if provided)

5.4 Interfaces (when applicable)

- screens/views (IDs)
- navigation groupings (if provided)
- per-screen actions mapped to capabilities

5.5 Operations (system actions)

- operations the system must perform (IDs)
- mapping: capability → operations (one-to-many)

5.6 Data Model (when applicable)

- data objects/entities (IDs)
- fields + types
- relationships

- sensitivity flags
- lifecycle states

5.7 Rules & Constraints

- business rules
- validation rules
- “must always / must never”
- compliance/security constraints (from standards snapshot)
- performance/reliability constraints (from NFRs if present)

5.8 Integrations (when applicable)

- integration objects (IDs)
- purpose, data in/out, triggers
- security/credential handling constraints

5.9 Verification Targets

- acceptance criteria statements (if present)
- “definition of done” and required checks (from standards snapshot + user)

5.10 Unknowns

- explicit unknown items recorded as unknown, with:
    - what is unknown
    - impact area (feature/data/auth/etc.)
    - whether blocking vs non-blocking *(policy choice)*

---

6) Structural Rules (anti-drift invariants)

6.1 Stable IDs

Every major object gets a stable ID:

- roles, capabilities, workflows, screens, operations, data objects, rules, integrations

6.2 Single Definition / No Duplicate Truth

- each object defined once
- everything else references it by ID

6.3 Referential Integrity

- references must resolve (workflow actor role exists, screen actions map to capabilities, etc.)

6.4 Explicit Unknowns

- unknowns must be explicit objects, not filled guesses

6.5 Standards Binding

- any decision constrained by the resolved standards must be recorded as a constraint in the spec (so it doesn't get re-decided later)

---

7) Mapping Outputs (key internal indexes)

Spec should include index maps like:

- `roles_by_id`, `capabilities_by_id`, `workflows_by_id`
- `screens_by_id` (if applicable)
- `data_objects_by_id` (if applicable)
- cross-maps:
  - `workflow_to_capabilities`
  - `capability_to_operations`
  - `screen_to_actions_to_capabilities`

These make decomposition deterministic.

---

8) Failure Modes / Edge Cases

- Feature list exists but workflows don't connect to features (spec builder must link or flag as unknown mapping)
- Roles inconsistent across sections (must unify)
- User provides screens but no actions (spec may add placeholder actions marked unknown, or require completion)
- Conflicting constraints (must surface contradiction object)

---

9) Interfaces

- **Work Breakdown generator** consumes Canonical Spec to create tasks
- **Acceptance Map builder** consumes Spec + verification rules
- **Template filler** references spec IDs and constraints
- **State Snapshot** references spec/work units by ID for continuity

Work Breakdown

A2.4 — Work Breakdown (Artifact Outline)

1) Artifact Name

Work Breakdown (decomposed build plan / work units)

2) Function

Transform the Canonical Spec into an ordered set of **work units** that an agent can execute step-by-step, with:

- clear scope per unit
- explicit dependencies
- traceability back to spec IDs
- sequencing that supports small verifiable increments

This answers: **"what to build, in what order."**

---

3) Inputs

- **Canonical Spec**
  - roles/capabilities/workflows/interfaces/data/rules/integrations
  - all stable IDs and cross-maps
- **Resolved Standards Snapshot**
  - constraints that affect sequencing (auth required early, testing standards, etc.)
- **(Optional) System build strategy rules**
  - e.g., "auth before feature work," "data model before endpoints," etc.

---

4) Outputs

A single **Work Breakdown** artifact:

- `work_breakdown_id`
- `submission_id`
- `spec_id`
- `units[]` (ordered list of work units)
- `dependency_graph` (unit-to-unit dependencies)
- `created_at`

---

5) Work Unit Structure (canonical fields)

Each work unit should include:

5.1 Identity

- `unit_id` (stable)
- `title`
- `description`

5.2 Scope Definition

- `scope_refs[]` (list of spec IDs this unit covers)
  - capabilities/workflows/screens/operations/data/rules/etc.
- `in_scope` (summary)
- `out_of_scope` (explicit exclusions)

5.3 Dependencies

- `depends_on[]` (unit_ids)
- `unblocks[]` (unit_ids) *(optional)*

5.4 Deliverables

- `deliverables[]` (what must exist after the unit)
  - e.g., “auth endpoints implemented,” “screen X built,” “data object Y created”
- `artifact_refs[]` (what artifacts must be updated, if relevant)

5.5 Verification Hooks (links to Acceptance Map)

- `acceptance_refs[]` (acceptance criteria IDs)
- `required_checks[]` (build/test/lint/etc.)

5.6 Risk / Notes (optional)

- `risk_flags[]` (e.g., “touches auth,” “migration risk,” “high coupling”)
- `notes`

---

6) Decomposition Rules (how units are created)

6.1 Partitioning strategy

Work is partitioned by a consistent strategy such as:

- foundational first (project setup, auth, core data model)
- then vertical slices (workflow-by-workflow, feature-by-feature)
- then integrations and polish

6.2 Unit size discipline

- units should be small enough to verify independently
- avoid “one unit per entire app”
- avoid splitting so small it becomes noise

6.3 Traceability requirement

Every unit must reference at least one spec ID (no “floating tasks”).

6.4 Deterministic sequencing

Given the same canonical spec + standards version, the work breakdown should be reproducible.

---

7) Failure Modes / Edge Cases

- Unit references spec IDs that don’t exist (breaks traceability)
- Dependencies form a cycle (must detect and resolve)
- Work units too broad (agent drifts), too narrow (inefficient)
- Missing foundational units (e.g., auth required but not scheduled early)

---

8) Interfaces

- **Acceptance Map** attaches criteria/checks per unit
- **State Snapshot** tracks progress per unit_id
- **Template filler** can generate step-by-step guidance from unit ordering

Acceptance Map

Acceptance Map (Artifact Outline)

1) Artifact Name

Acceptance Map (definition of “done” per work unit)

2) Function

Define measurable completion rules for each work unit so the agent cannot claim progress without evidence. This artifact converts “build tasks” into “build tasks with proof requirements.”

This answers: **“how do we know each unit is done?”**

---

3) Inputs

- **Work Breakdown**
  - unit list + dependencies + deliverables
- **Resolved Standards Snapshot**
  - required checks (build/lint/tests), quality bar, security expectations
- **Canonical Spec**
  - acceptance criteria from intent/DoD (if provided)
  - constraints that must be validated (auth, permissions, etc.)

---

4) Outputs

A single **Acceptance Map** artifact:

- `acceptance_map_id`
- `submission_id`

- `spec_id`
- `work_breakdown_id`
- `acceptance_items[]` (criteria objects)
- `unit_acceptance_index` (unit_id → acceptance item IDs)
- `created_at`

---

5) Acceptance Item Structure (canonical fields)

Each acceptance item should include:

5.1 Identity

- `acceptance_id` (stable)
- `unit_id` (required link)

5.2 Criterion

- `statement` (clear “done when…” requirement)
- `type` (enum-like category)
    - examples: build_check, test_check, functional_check, security_check, ui_check, data_integrity_check, regression_check

5.3 Proof Requirement

- `proof_type`
    - command_output, test_result, screenshot, log_excerpt, diff_reference, manual_checklist

- `proof_location`
    - where evidence should be recorded/stored (path or reference key)
- `proof_required` (boolean; default true)

5.4 Verification Instructions

- `how_to_verify` (short instructions: command to run, flow to test)
- `expected_result` (what passing looks like)

5.5 Severity / Gating

- `gating` (hard gate vs soft gate)
    - hard_gate: must pass to mark unit complete
    - soft_gate: warning-level quality check

---

6) Map Rules (how acceptance is generated)

6.1 Minimum acceptance per unit

Every unit must have at least:

- 1 deliverable confirmation criterion
- 1 verification/proof criterion

6.2 Standards binding

Acceptance criteria must include required checks from standards snapshot where applicable:

- build must pass

- lint must pass (if required)
- tests must pass (if required)

6.3 Risk-based enrichment

If unit touches high-risk areas (auth, payments, migrations), include additional acceptance items:

- permissions checks
- regression checks
- error handling checks

6.4 Traceability

Acceptance items must reference:

- the unit_id
- and indirectly the spec scope through the unit's scope_refs (no orphan acceptance)

---

7) Failure Modes / Edge Cases

- Acceptance criteria too vague ("works well") → not verifiable
- Proof requirement missing → agent can claim completion without evidence
- Checks not runnable in environment → must specify fallback proof type
- Overloading every unit with too many checks → slows execution

---

8) Interfaces

- **Agent execution** uses acceptance map to verify each unit
- **State Snapshot** stores pass/fail status and proof pointers per acceptance_id
- **Final completion** aggregates hard gates across all units

State Snapshot

State Snapshot (Artifact Outline)

1) Artifact Name

State Snapshot (resumable execution state / continuity record)

2) Function

Maintain a single, structured, up-to-date record of **where the build is**, so work can pause/resume (or hand off to another agent) without reinterpreting intent.

This answers: **“what’s done, what’s next, what’s blocked, and what proof exists?”**

---

3) Inputs

- **Work Breakdown**
  - `units[]`, dependencies
- **Acceptance Map**
  - acceptance items per unit + proof requirements
- **Agent progress events**
  - unit started/completed
  - acceptance checks run (pass/fail)
  - proof artifacts captured
- **(Optional) User/owner decisions**
  - scope changes, overrides, resolved unknowns

---

4) Outputs

A single **State Snapshot** artifact that is updated over time:

- `state_id`
- `submission_id`
- `spec_id`
- `work_breakdown_id`
- `acceptance_map_id`
- `current_unit_id`
- `status` (in_progress / blocked / complete)
- `updated_at`
- `history[]` *(optional event log or pointer to event log)*

---

5) State Snapshot Structure (canonical fields)

5.1 Progress Tracking

- `unit_status[]` (per unit)
  - `{ unit_id, status: not_started|in_progress|blocked|done, started_at?, completed_at? }`
- `completed_units[]` (optional derived list)

5.2 Acceptance Tracking

- `acceptance_status[]` (per acceptance item)
  - `{ acceptance_id, unit_id, status: not_run|pass|fail, last_run_at?, proof_refs[] }`

5.3 Proof Index

- `proof_artifacts[]`
  - `{ proof_id, acceptance_id, type, location, summary?, created_at }`

5.4 Blockers / Unknowns

- `blockers[]`
  - `{ blocker_id, scope_ref? (spec ID), description, severity, created_at, resolved_at? }`
- `open_unknowns[]`
  - `{ unknown_id, scope_ref? (spec ID), description, impact, status: open|resolved }`

5.5 Decisions / Overrides (traceability)

- `decisions[]`
  - `{ decision_id, description, rationale?, made_by, made_at, scope_ref? }`
- `overrides[]`
  - `{ override_id, field_path, before, after, reason, approved_by, approved_at }`

5.6 Last Known Good (LKG)

- `last_known_good`
  - `{ unit_id?, verification_summary, proof_refs[], timestamp }`

---

6) Update Rules (when snapshot must change)

State snapshot must update when:

- a unit starts or completes
- an acceptance check runs (pass/fail)
- a proof artifact is generated/recorded
- a blocker/unknown is created or resolved
- a decision/override is approved

---

7) Invariants

- There is one current “latest” snapshot (or a latest pointer)
- Unit completion requires all **hard-gate** acceptance items passing
- Proof references must point to real stored artifacts/paths
- Snapshot must remain consistent with unit dependency ordering (cannot mark dependent unit done before prerequisites unless explicitly overridden and recorded)

---

8) Failure Modes / Edge Cases

- Agent claims completion without updating acceptance/proof → snapshot must not mark done
- Proof exists but not linked to acceptance_id → becomes unusable
- Scope change midstream → must create decision + update affected units/acceptance criteria, not silently proceed

- Multiple agents editing state concurrently → needs a merge policy (single writer or event log approach)

---

9) Interfaces

- **Agent** reads snapshot first to resume work deterministically
- **Verification** uses snapshot proof index to audit completion
- **Handoffs** rely on snapshot to prevent continuity errors

Libraries

Libraries (reusable stores)

Purpose: reusable building blocks that keep outputs consistent across projects.

11. **Template Library**

- Contains: all template definitions (structure + required fields + fill rules).
- Organized by: template type and use-case.
- Output: template set used by the filler.

12. **Standards Library**

- Contains: design/engineering/security/quality standards.
- Organized by: standard category + version.
- Output: inputs to the standards resolver.

13. **Vocabulary/Definitions Library**

- Contains: controlled language (terms, actions, tags/labels).
- Output: normalization rules + consistent phrasing/IDs.

14. **Example/Reference Library (optional)**

- Contains: canonical examples, reference patterns, sample outputs.
- Output: guide rails for generation and consistency.

Template Library

Template Library — Spec Outline (Final Types, Senior-Level Coverage)

1) Purpose

The Template Library is the **versioned collection of structured templates** the internal agent fills to generate a complete, consistent Agent Kit. It exists to:

- prevent missing key project parts (senior/dev-team coverage)
- prevent drift (consistent structure + required fields + gates)
- standardize output quality across projects and agents

---

2) Scope

Covers templates for the **final merged template types**:

1. Product / Requirements
2. Design
3. Architecture
4. Implementation *(includes planning)*
5. Security & Access
6. Quality *(includes a11y + performance checks)*
7. Operations *(includes support/runbooks/incident)*
8. Data *(conditional)*
9. API / Interface Contracts *(conditional)*
10. Release *(conditional)*

11. Governance / Change Control *(conditional)*
12. Analytics / Telemetry *(conditional)*

---

3) Library Structure (how it's organized)

The library is organized into:

3.1 Template Type Folders

One folder per template type, containing one or more templates.

3.2 Template Index (Registry)

A machine-readable registry that lists every template and its metadata:

- `template_id` (stable)
- `title`
- `type` (one of the 12 types)
- `applies_when` (routing/gates: category/type preset/audience/build_target)
- `requiredness` (always / conditional)
- `required_by_skill_level` (Beginner/Intermediate/Expert rules)
- `inputs` (what it needs from Canonical Spec + Standards Snapshot)
- `outputs` (what filled doc it produces)
- `upstream_dependencies` (must be valid before this template can be generated)
- `compliance_checks` (template-level gates)
- `version`

---

4) Template File Contract (what every template must contain)

Every template must include these sections in this order:

1. **Header**

- Template ID, Title, Type, Version

2. **Purpose**

- One paragraph describing what this template does

3. **Inputs Required**

- Explicit list of required upstream artifacts/fields (by canonical keys/IDs)

4. **Required Fields**

- What must be filled for the template to be considered valid

5. **Optional Fields**

- What improves quality but isn't mandatory

6. **Rules**

- "Must/never" rules inside this template
- No duplicate truth rule: reference Canonical Spec IDs, don't redefine

7. **Output Format**

- Required structure, headings, tables/lists formats
- Placeholder conventions (how variables look)

8. **Cross-References**

- What other templates/artifacts this links to by ID

9. **Unknown Handling**

- How to represent unknowns (explicit UNKNOWN blocks)

10. **Compliance Gate**

- Checklist that must pass before moving to downstream templates

---

5) Placeholder / Variable Convention (how templates get filled)

Templates use placeholders that map to:

- **Direct inserts** from Canonical Spec (roles, features, workflows, data objects)
- **Derived inserts** (workflow→screens mapping, capability→operations mapping)
- **Standards inserts** (design rules, stack rules, security rules)

Required rules:

- placeholders must not require “creative guessing”
- unresolved placeholders must become explicit `UNKNOWN` entries, not silent fill

---

6) Depth Profile Rules (skill_level behavior)

Template Library must support profile-based requirements:

- **Beginner:** fewer required sections; more UNKNOWN allowed; more scaffolding text
- **Intermediate:** standard required sections; moderate UNKNOWN
- **Expert:** advanced sections required when applicable; UNKNOWN rare and justified

This is encoded per template in the Template Index via:

- `required_by_skill_level`
- (optional) template variants if needed

---

7) Template Selection Rules (how internal agent chooses templates)

Selection is deterministic based on:

- routing (category/type preset/audience/build_target)
- gates (data enabled, auth required, integrations enabled, etc.)
- skill level requirements

Output is the **Template Set** for the project (the exact list of templates that must be filled).

---

8) Compliance & Build Order (critical)

The library includes a **build order contract**:

- each template declares `upstream_dependencies`
- the internal agent cannot generate the next template until upstream templates pass their compliance gates
- compliance gates must be checkable (no vague “looks good” gates)

---

9) Versioning

- The library has a `template_library_version`
- Each template has its own `template_version`
- Every generated kit stamps:
    - library version
    - template versions used

- Changes must be tracked with compatibility notes (what breaks old kits)

---

10) Definition of Done (for the Template Library)

The Template Library is complete when:

- every template type has at least one template
- every template is registered in the Template Index
- every template follows the Template File Contract
- build order + compliance gates are defined and non-contradictory
- selection rules cover all routing presets and gates

Tab 41

Template Library — Final Type List + Templates

1) Product / Requirements

- PRD-01 Product Requirements Document
- PRD-02 Personas, Roles, Permissions Summary
- PRD-03 Feature Catalog (Must/Nice/Future)
- PRD-04 Workflow Narratives (Top Use Cases)
- PRD-05 Scope Boundaries (Non-goals + Constraints)
- PRD-06 Success Criteria + Definition of Done

2) Design

- DES-01 Design Direction + Brand Snapshot *(includes voice/copy rules)*
- DES-02 Layout System + Typography Rules
- DES-03 Component Rules + Usage Standards
- DES-04 Screen Layout Specs *(screen-by-screen)*
- DES-05 Interaction, States, Motion Rules
- DES-06 Accessibility Requirements *(design-side)*

3) Architecture

- ARC-01 System Overview + Boundaries
- ARC-02 Module Responsibilities + Interfaces
- ARC-03 Data Flow + Integration Points
- ARC-04 Key Design Decisions (ADR Pack)

- ARC-05 Runtime Topology Overview *(if applicable)*

4) Implementation (includes Planning)

- IMP-01 Build Order + Milestones
- IMP-02 Work Units Execution Guide *(how to run units)*
- IMP-03 Repo/File Structure Contract
- IMP-04 Coding Conventions + Patterns
- IMP-05 Feature Slice Implementation Guide
- IMP-06 Integration Implementation Guide *(if integrations)*
- IMP-07 Migration/Upgrade Guide *(if existing project)*

5) Security & Access

- SEC-01 Authentication + Sessions Spec
- SEC-02 Authorization / Permissions Enforcement Spec
- SEC-03 Data Handling + Privacy Rules
- SEC-04 Abuse Controls + Rate Limits *(if public-facing)*
- SEC-05 Security Verification Checklist

6) Quality (includes a11y verification + performance)

- QA-01 Acceptance Criteria Map *(per work unit)*
- QA-02 Test Plan (Unit/Integration/E2E)
- QA-03 Verification Commands + Proof Requirements
- QA-04 Regression Checklist

- QA-05 Performance & Reliability Checks
- QA-06 Accessibility Verification Checklist *(test-side)*

7) Operations (includes support/runbooks/incidents)

- OPS-01 Deployment + Environments Plan
- OPS-02 Configuration + Secrets Handling
- OPS-03 Observability Plan (Logs/Metrics/Alerts)
- OPS-04 Backup/Recovery + Rollback Plan
- OPS-05 Runbook (Operational Procedures)
- OPS-06 Incident Response + Postmortem Process

8) Data (conditional)

- DATA-01 Data Model Overview
- DATA-02 Entity + Field Dictionary
- DATA-03 Relationships + Ownership/Tenancy Rules
- DATA-04 Migrations + Backfill Plan
- DATA-05 Retention / Export / Deletion Rules

9) API / Interface Contracts (conditional)

- API-01 API Contract Overview
- API-02 Endpoint Catalog (Req/Res/Error)
- API-03 Event/Webhook Contracts
- API-04 Error Model + Status Codes

- API-05 Versioning + Compatibility Rules

10) Release (conditional)

- REL-01 Release Plan
- REL-02 Rollout Strategy (Flags/Phasing)
- REL-03 Rollback Plan
- REL-04 Release Notes Template
- REL-05 Launch Checklist

11) Governance / Change Control (conditional)

- GOV-01 Change Control Policy
- GOV-02 Versioning + Release Discipline
- GOV-03 Deprecation + Migration Policy
- GOV-04 Decision Log (ADR Index)
- GOV-05 Locked Surface / Non-Change Rules

12) Analytics / Telemetry (conditional)

- ANALYTICS-01 Metrics + KPI Definitions
- ANALYTICS-02 Event Tracking Plan
- ANALYTICS-03 Funnel / Journey Measurement Plan
- ANALYTICS-04 Dashboard Requirements

1.

Generators

Kit Deliverable

Control Assets

AGent Worklog bot

Trigger Monitoring Outline (Internal Agent)

1) Trigger Categories (what we monitor)

- **Unit lifecycle**: `unit.started`, `unit.blocked`, `unit.completed`
- **Acceptance/verification**: `acceptance.check.run`, `acceptance.check.pass`, `acceptance.check.fail`, `proof.attached`
- **Gates/overrides**: `gate.pass`, `gate.fail`, `override.applied`
- **Artifact changes**: `artifact.changed`
- **Commands/console**: `command.run`, `command.exit`

---

2) Data to Pull Per Trigger (authoritative only)

A) Unit lifecycle triggers

Pull

- `unit_id`
- Work Breakdown: unit details (scope_refs, deps, deliverables, acceptance_refs)
- State Snapshot: current status for unit + current_unit_id
- Blockers linked to unit
  Update
- State Snapshot (unit status + timestamps + current_unit_id + blockers)

B) Acceptance/verification triggers

Pull

- `acceptance_id`
- Acceptance Map: criterion, gating (hard/soft), proof requirements, verification instructions
- Work Breakdown link: `unit_id`
- Latest command context (if relevant): cmd, cwd, exit code, stdout/stderr pointer

Update

- State Snapshot (acceptance status pass/fail + last_run_at)
- Proof Log (proof_id + acceptance_id + unit_id + location + timestamp)

C) Gate/override triggers

Pull

- gate_id + target artifact
- Gate report: issues + pointers + remediation
- Version stamps (schema/standards/templates/system)

Update

- Gate report log (append-only)
- Override record (if applied)
- State Snapshot decision/override pointer (if impacts execution)

D) Artifact change triggers

Pull

- changed paths
- before/after hash or diff summary
- artifact identity/type (spec/plan/template/state/etc.)
- version stamps

Update

- Change log entry
- If artifact is supposed to be immutable: raise gate violation

E) Command/console triggers

Pull

- command, cwd, environment label (if known)
- exit code
- stdout/stderr pointers (bounded)
  Update
- Proof candidates (when mapped to acceptance_id)
- State Snapshot activity / last-known-good (when checks prove compliance)

---

3) Universal Context (pulled on every trigger)

- `submission_id`
- `spec_id`
- `work_breakdown_id`
- `acceptance_map_id`
- `kit_id`
- `current_unit_id`
- versions: schema / standards / templates / system

---

4) Hard Rule

Triggers must pull **only authoritative artifacts** (spec/plan/acceptance/state/gate reports/proof logs/command outputs).
No chat memory as truth.

Axion More Code Update

Yes — **some code can make Axion materially better** (faster, more consistent, less drift), but only in specific places where “docs alone” can’t enforce contracts. Think of code as the **enforcement + automation layer** around your docs.

Where code adds the most value

1) Validator + Dependency Engine

Turn Intake Schema + dependency rules + skill-level thresholds into an actual deterministic validator.

- Benefit: stops bad inputs early, fewer downstream contradictions.

2) Normalizer (structured extraction + canonicalization)

Automate:

- enum normalization
- deduping lists
- canonical naming (roles/features/workflows)
- safe list extraction from free-text
- Benefit: less manual cleanup, cleaner canonical spec.

3) Spec Builder (ID assignment + referential integrity)

Code that:

- assigns stable IDs
- builds cross-maps (workflow→features, role→permissions, etc.)
- validates references
- Benefit: prevents “duplicate truth” and broken references.

4) Template Filler (placeholder resolution + derived inserts)

A filler that:

- resolves placeholders from canonical spec + standards snapshot
- computes derived inserts (feature→screens→operations)
- enforces “no creative guessing” by producing UNKNOWN blocks
- Benefit: consistent filled docs; minimal agent reasoning.

5) Gate Runner (compliance automation)

A runner that executes gates and outputs gate reports:

- template completeness checks
- kit layout checks
- reference integrity checks
- version pin checks
- Benefit: “no forward progress without compliance” becomes real.

6) Packager (kit file tree + manifest + zip)

Automate folder creation, template placement, N/A stubs, and manifest/index generation.

- Benefit: predictable kits, fewer missing files, faster runs.

7) Change Detection + Version Stamping

Code that:

- hashes artifacts
- stamps versions into the kit
- detects “self-modifying run” or unversioned changes
- Benefit: stronger determinism + governance.

8) Trigger/State Automation Bot (if you want it)

A small bot that listens for:

- unit/acceptance/gate events
- command outputs
  and updates state snapshot + proof log.
- Benefit: less reliance on chat memory; better continuity.

What usually should *stay* as docs

- the SYS/STD/TMP contracts themselves
- template content
- runbooks and human-readable explanations

So: more code helps most when it **enforces** or **automates** the contracts; it doesn’t help when it just duplicates what the docs already say.

Tab 43

1) Intake Surface

Artifacts needed:

- **INTAKE_FORM_CONFIG** (page/module layout + routing map + form_version)
- **INTAKE_SCHEMA** (canonical schema object + schema_version)
- **INTAKE_RULESET** (INT-03 rules list: rule_id/when/then/error_code/pointers)
- **ENUM_REGISTRY** (all enums referenced by schema + UI)
- **VALIDATION_REPORT** (validator output format: pass/fail + rule hits)

2) Canonical Specification Model

Artifacts needed:

- **CANONICAL_SPEC_SCHEMA** (the authoritative “canonical spec” shape)
- **SPEC_INDEX** (screen/endpoint/entity/business_rule/component/acceptance_criterion/test_case/flow entries)
- **REFERENCE_RULES / POINTER_MODEL** (how refs are written/resolved across items)
- **CANONICAL_SPEC_EXAMPLE** (golden example payload that validates)

3) Template System

Artifacts needed:

- **TEMPLATE_INDEX / REGISTRY** (template IDs, versions, required sections, outputs mapping)
- **TEMPLATE_FILES** (the actual templates: TMP-* as concrete files)
- **TEMPLATE_COMPLETENESS_RULES** (objective checks per template: required headings/fields)

- **TEMPLATE_RENDER_OUTPUT_FORMAT** (how filled templates are stored/versioned)

4) Knowledge & Standards Library

Artifacts needed:

- **STANDARDS_INDEX** (STD registry: IDs, titles, scope, applicability rules)
- **STANDARDS_ENTRIES** (the actual STD/KSL documents as files)
- **CONSTRAINT_PACKS** (locked rulesets the build must obey: naming, routing, UI, security, etc.)
- **APPLICABILITY_MAP** (which standards apply by category/type/skill/build_target)

5) Pipeline Orchestration

Artifacts needed:

- **PIPELINE_DEFINITION** (stage order, inputs/outputs, gating points)
- **STAGE_IO_CONTRACTS** (for each stage: consumes/produces)
- **RUN_PLAN / EXECUTION_PLAN** (what will run for this project + why)
- **STAGE_REPORTS** (per stage output summaries, including pointers to produced artifacts)

6) Assembly & Packaging

Artifacts needed:

- **PACKAGING_MANIFEST** (what's included in a kit/package + versions/hashes)
- **FOLDER_TREE_SPEC** (authoritative kit structure)
- **BUNDLE_OUTPUTS** (the zipped kit / domain pack / "handoff bundle" outputs)

7) Agent Instruction Layer

Artifacts needed:

- **ALRP/TIES RUNTIME PACK** (the exact agent instruction payload used for a run)
- **ROLE_PROMPTS / ROLE_CARDS** (per-agent role prompt artifacts, versioned)
- **CONTEXT_MAP / INJECTION_MAP** (what gets injected where, deterministic)

8) Execution Contracts

Artifacts needed:

- **EXEC_CONTRACT_FILES** (the “must follow” execution rules as machine-checkable policy)
- **COMMAND_SET / TOOLING_ADAPTERS** (the allowed commands, tool bindings, IO paths)

9) Verification & Proof

Artifacts needed:

- **GATE_REPORTS** (per gate: results + rule IDs + pointers + proof refs)
- **PROOF_LEDGER** (append-only log of checks + outputs + hashes)
- **VERIFICATION_ARTIFACTS** (test outputs, lint outputs, build logs, screenshots if applicable)

10) State & Continuity

Artifacts needed:

- **STATE_SNAPSHOT** (what is locked, what is open, what was produced, next actions)
- **RESUME_PACKET** (minimum required artifacts to resume deterministically)

- **CHANGELOG / DIFF_RECORD** (what changed between runs)

11) Versioning & Governance

Artifacts needed:

- **VERSION_REGISTRY** (doc/template/schema versions + compatibility notes)
- **OVERRIDE_RECORDS** (who overrode what gate, why, and scope)
- **POLICY_CONFIG** (severity policy, allowed exceptions)

12) Tooling & Platform Integration

Artifacts needed:

- **INTEGRATION_CONFIG** (repo mapping, paths, environment assumptions)
- **ADAPTER_SPEC** (how Axion maps to Replit/GitHub/etc. in practice)
- **BOOTSTRAP_SCRIPT / QUICKSTART** (the “do this first” runnable artifact)